

Trees, Walks, and Words: A Logic for File System Interactions

ERIC ZHAO, Brown University, USA

ZEKAI LI, Brown University, USA

LUKAS LAZAREK, Brown University, USA

FEIYANG WANG, Brown University, USA

KONSTANTINOS KALLAS, University of California, Los Angeles, USA

ROBERT LEWIS, Brown University, USA

NIKOS VASILAKIS, Brown University, USA

Programs often interface with file systems for a variety of tasks, from retrieving and storing computation data to configuring the operating system. These interactions are mediated by *paths*, and the complex interplay between programs, paths, and hierarchical file system state can lead to unexpected behaviors ranging from silent failures to irreversible data loss and security vulnerabilities. Yet, this domain remains underserved by formal reasoning. This paper addresses that gap by developing foundations for automated formal analysis of file-system-interacting programs. Grounding our study in a survey of APIs across several widely used standard libraries, we distill core file system behaviors into a representative object language, `WHILEFS`. We then develop a theory of *tree-walking word equations* (TWE) and show that it is adequate to reason about file system interactions via sound and complete symbolic semantics for `WHILEFS`. We demonstrate that the satisfiability question for TWE is as hard as the longstanding open problem of word equations with length constraints. Nonetheless, we also show that two useful fragments remain decidable in PSPACE by reduction to standard word equation theories. Together, these results demonstrate that TWE and `WHILEFS` constitute a rich foundation for formal reasoning about programs that interact with file systems.

1 INTRODUCTION

Interactions with file systems are pervasive in software programs, which routinely read from and write to files and directories to store and retrieve user data, cache intermediate computations, read configuration files, and interface with other systems resources such as I/O channels and devices. File systems are used across every layer of the software stack, from shell scripts and package managers to compilers, databases, and web servers, and the standard libraries of most programming languages expose dedicated APIs for file system operations.

The interplay between program execution, file system state, and the *paths* used to mediate between them is often subtle and gives rise to substantial complexity in reasoning about these programs. As a result, developers and agents often fail to reason effectively about the file system, producing and executing software with bugs ranging from silent failures to dangerous behaviors such as irrecoverable data loss. Indeed, file-system-related subtleties underlie notable catastrophic incidents, such as when updating Steam installations on Linux inadvertently deleted all user-writable data on affected machines [57], or when an agentic coding tool asked to organize a user’s desktop instead deleted an archive of family photographs [33]. Misuse of the file system can also expose security vulnerabilities; thousands of known CVEs in high-profile software are linked to improper file system path handling [43, 44], such as the “Zip Slip” vulnerability [53]. Despite the evident challenge and irreversible consequences of such bugs, this domain remains underserved by formal reasoning techniques: most prior work is limited to specific kinds of programs (e.g., no file system

Authors’ addresses: [Eric Zhao](#), Brown University, Providence, USA; [Zekai Li](#), Brown University, Providence, USA; [Lukas Lazarek](#), Brown University, Providence, USA; [Feiyang Wang](#), Brown University, Providence, USA; [Konstantinos Kallas](#), University of California, Los Angeles, Los Angeles, USA; [Robert Lewis](#), Brown University, Providence, USA; [Nikos Vasilakis](#), Brown University, Providence, USA.

modifications, or no first-class path values) or does not reason about the file system’s hierarchical structure (§7).

In this work, we introduce a framework for formal reasoning of file system interactions, laying the theoretical foundations for automated analyses. The key elements are (1) a model of the file system and programs interacting with it (expressed as an object language WHILE_{FS}), and (2) a theory of *tree-walking word equations* (TWE) tailored for reasoning over our model, by expressing fundamental facts and queries about possible file system states during program execution. We ground the framework and WHILE_{FS} by conducting a survey of the file system APIs of the standard libraries of four widely used languages (C, Python, Node.js, and Rust) and using its results to decide what to include and what to leave out of WHILE_{FS} (§3). We demonstrate that the theory of tree-walking word equations (TWE) is a suitable base theory of paths and file systems by giving a symbolic semantics for WHILE_{FS} , capturing the space of possible file system states and behaviors using TWE-formulae. We show the symbolic semantics sound and complete with respect to concrete executions.

We then study the decidability of TWE-satisfiability. We demonstrate that the theory of word equations with length constraints (WE + LEN)—the decidability of which is a longstanding open problem [11, 21]—can be reduced to TWE (§5). Although the full logic’s decidability is therefore an open question, we also show that two restricted yet useful fragments of the logic are decidable. In particular, we formulate and prove correct decision procedures for *linear* and *bounded non-linear* fragments of TWE (§6). The former suffices to reason about programs which exhibit only simple downward path traversals, while the latter fragment permits a bounded degree of upward path movement. Since upward path traversals are often limited in practice, these fragments provide a starting point for rigorous automated analyses of common classes of programs.

Contributions. We start with an informal exposition of paths and hierarchical file systems (§2). The following sections describe the paper’s contributions:

- A systematic survey of several real file system libraries and a representative object language WHILE_{FS} distilling core file system behaviors and interaction (§3).
- A quantifier-free theory of *tree-walking word equations* (TWE) that can be used to reason about WHILE_{FS} , as demonstrated with a sound and complete symbolic semantics (§4).
- A proof that the satisfiability question for TWE is at least as hard as that of word equations with length constraints, a longstanding open problem (§5).
- Proofs of decidability for two restricted yet useful fragments of TWE (§6).

We conclude with a discussion of related work (§7) and future directions (§8).

2 UNDERSTANDING PATHS AND HIERARCHICAL FILE SYSTEMS

In this section, we walk through three different perspectives on paths, each of which illustrates one additional facet of their complexity. First, we see paths as addresses in the file system, similarly to how pointers refer to cells in a program’s heap (§2.1). Second, we uncover path resolution, extending paths as procedures that walk the file system before reaching their destination, imposing constraints on the whole sequence of visited nodes (§2.2). Finally, we introduce programs into the mix, which manipulate paths as sequences, introducing sequence-level constraints in addition to file system constraints (§2.3).

<pre> rmr(path) $\triangleq$ 1 st := stat(path) 2 if !st then 3 return 4 5 if st.type == ● then 6 unlink(path) 7 else if st.type == ○ then 8 ds := opendir(path) 9 while ds != [] do 10 name, ds := readdir(ds) 11 rmr(join(path, name)) 12 rmdir(path) </pre>	<pre> rmr(/app) ① = stat(/app) [config, db] = opendir(/app) config, [db] = readdir([config, db]) rmr(/app/config) ③ = stat(/app/config) [] = opendir(/app/config) rmdir(/app/config) db, [] = readdir([db]) rmr(/app/db) ④ = stat(/app/db) unlink(/app/db) rmdir(/app) </pre> for config for db
--	--

Listing 1. A simplified implementation **rmr** of recursive deletion from busybox [12] (left) and the execution trace of **rmr**(/app) on the tree in figure 1 (right). The execution trace tracks the sequence of calls to file system APIs across recursive calls to **rmr**. Note that while **stat** actually returns a summary of the node at the specified location, for convenience we show the actual node (e.g., ①) from figure 1.

2.1 Paths are addresses

Paths locate data stored in the file system, similarly to how pointers locate memory in the heap. However, rather than a flat array the file system is usually hierarchical, resembling a tree of named locations such as the one shown in figure 1. Each location is either *absent* or has a *node*. Each node is a *directory* ○, which consists of named child *entries*, or a *file* ● at a leaf position, which holds data such as byte content. The tree starts at the *root directory* (① in figure 1) and a path identifies a location in the tree using a *sequence of names* separated by a delimiter (e.g., /).

Listing 1 (left) showcases paths as addresses through a simplified version of the implementation of `rm -r` from busybox [12], a popular implementation of Unix coreutils, that recursively deletes directories and files at a given path. Suppose that we call **rmr**(/app) for the file system tree in figure 1. On line 1, **stat** retrieves the node at path. In this case, since /app locates the directory ①, the program iterates through its entry names via a directory stream and recursively invokes **rmr** for each one. The program constructs the recursive call paths using the *path expression* **join**(path, name) that extends path with name.

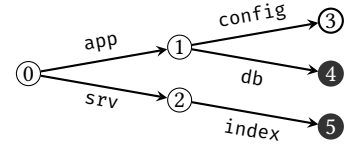


Figure 1. Tree representing a file system.

2.2 Paths are procedures

The address view captures the purpose of paths, but hides the actual process with which a path indexes in the file system. This process is called *path resolution* and instead of directly indexing a node, it recurses over the name sequence traversing tree nodes. That process can *fail*, if an intermediate node is not a directory but rather a file or absent. Resolution succeeds only if the entire path is processed without failure, implicitly constraining all *ancestor* nodes to be directories. For instance, successful resolution of /app/db implies that the nodes at / and /app are directories.

The picture is complicated by the *parent reference*, “..”, that walks up instead of down the tree: e.g., resolving .. from ③ in figure 1 traverses ①. Parent references introduce *aliasing*, since a node

can now be identified by infinitely many paths: ② in figure 1 is identified by `/srv`, `/app/./srv`, `/app/./app/./srv`, and so on. Note that resolution *saturates* when resolving `..` at the root directory: e.g., `/`, `/..`, and `/../..` all resolve to the root. File system APIs often provide “normalization” operations that eliminate parent references to derive a single *linear* path to a node. For example, `/app/./srv` can be normalized to `/srv` by eliminating the pair `app/..`. Note that syntactic normalization *does not* strictly preserve the behavior of paths, since it might hide a path resolution failure: e.g., while `/home/./srv` fails, its normalized version (`/srv`) *succeeds*.

In light of path resolution, a path can be understood as a *procedure*: while an address describes *where* a location is, a path dictates *how* to reach it by specifying a *walk* along the file system tree. For example, resolving `/app/./srv` on the tree in figure 1 yields the successful walk $① \rightarrow ① \rightarrow ① \rightarrow ②$. We call the last node the *target* and the preceding sequence the *footprint* of the walk.

Ignoring the procedural perspective of paths can lead to buggy programs. For example, the implementation in listing 1 does not account for paths as procedures, causing it to misbehave on inputs with `..` components. Consider an invocation `rmr(/app/config/..)` on the same tree from figure 1, which should remove ① just like `rmr(/app/)`. Until the second recursive call, the execution trace (depicted in listing 2) closely resembles the happy path. However, `stat(/app/config/./db)` fails, because resolving the path demands walking `config` in ①, but it was deleted by the first recursive call. The program silently fails and leaves ① partially removed.

```
rmr(/app/config/..)
① = stat(/app/config/..)
[config, db] = opendir(/app/config/..)
config, [db] = readdir([config, db])
rmr(/app/config/./config)
③ = stat(/app/config/./config)
[] = opendir(/app/config/./config)
rmdir(/app/config/./config)
db, [] = readdir([db])
rmr(/app/config/./db)
✗ = stat(/app/config/./db)
```

Listing 2. The failing execution trace for `rmr(/app/config/..)` on the tree in figure 1.

Improper accounting for walks can also lead to security vulnerabilities, such as bypassing access control (see CWE-22 [43] and 23 [44], which record over 2,700 high or critical severity CVEs). A common pattern of such bypass happens when the program enforces access control by checking path prefixes (e.g., restricting access to `/srv` by checking `isPrefix(/srv, path)`), which can be bypassed using parent references (e.g., `/srv/./app/db`).

2.3 Paths are dynamic

An additional layer of complexity is that programs can arbitrarily manipulate paths, by building them up from sequences (e.g., `join`), decomposing, comparing (e.g., `isPrefix`), and passing them around as ordinary data. To statically reason about a program therefore means to reason about constrained (or unconstrained) sets of paths, how they are transformed by sequence operations, and whether the walks they describe on the possible sets of file system states fail or succeed.

Listings 1 and 2 illustrate this complexity concretely through the program’s combination of control, syntactic path construction, and file system mutation. The behavior of the program depends on the combination and interaction of constraints across paths, walks, file system states, and control flow. The bug can only be identified by simultaneously tracking constraints at (1) the syntactic level (the construction of `/app/config/./config` via `join`), (2) file system state (the absence of `/app/config` on the second iteration), (3) control flow (the loop over entries of the directory), and (4) walks (which fail for `/app/config/./db` once `/app/config` is deleted).

In the rest of the paper, we develop foundations to formally reason about such programs and their intertwined complexity. Indeed, we model these kinds of programs (§3) and show that the bug above can be witnessed by symbolic execution traces based on satisfiability of formulae in the theory of TWE (§4). The satisfiability question is hard in general (§5) yet decidable in cases of bounded upwards traversal such as this one (§6). Note that we have avoided addressing many additional features present in real file systems, such as links, mounts, overlays, and access control. Even without these, we see significant fundamental complexity in reasoning about file system interactions, so we defer the study of such features for future investigation.

3 SURVEY AND CORE OF FILE SYSTEM INTERACTIONS

To ground formal analysis, we first conduct a systematic survey of file-related APIs across the standard libraries for several widely used programming languages (§3.1). The survey reveals a stable *core* of abstractions and operators via which programmers interface with the file system (§3.2), and this core informs the design of an archetypal object language WHILE_{FS} (§3.3) which will serve as the basis for the remainder of the paper.

3.1 Survey

We survey the standard libraries of  glibc 2.41,  Python 3.13,  Node.js 24.1, and  Rust 1.96, four widely used languages with well-documented APIs, examining official documentation for abstractions and operations which interface with the file system. From the individual APIs we identify atomic behaviors, organize them into loose families, and distinguish the data abstractions α on which they operate. Note that a given API function may couple several different kinds of behaviors, and each behavior may operate on several different data abstractions. For instance, `fopen`  may both open and create files. We do not consider concurrency (e.g., atomicity and thread-safety), synchronicity (e.g., blocking vs asynchronous execution), or platform-specific variation (e.g., case-sensitivity and other conventions) and leave these aspects for future work. We also abstract over differences in error-handling.

The complete results comprise 524 file system-related APIs capturing 128 distinct behaviors operating on 20 data abstractions across 33 behavior families. Table 1 gives a compact summary of these behavior families, collected loosely into 4 groupings; the complete results can be found in the supplementary material (Appendix A).

Paths. Locations in the file system are addressed by *paths* $\mathbb{P}$, which are sequences of components. The libraries provide specific operators to manipulate paths syntactically: e.g., composition primitives concatenate paths with the delimiter (e.g., `PurePath.joinpath` $\mathbb{P}$, `path.join` $\mathbb{N}$), while decompositions extract the last component (e.g., `basename` $\mathbb{G}$, `PurePath.name` $\mathbb{P}$, `path.basename` $\mathbb{N}$) or the prefix sub-path (e.g., `dirname` $\mathbb{G}$, `PurePath.parent` $\mathbb{P}$, `path.dirname` $\mathbb{N}$). More complex decompositions may further extract stems or extensions from the basename (e.g., `PurePath.stem` $\mathbb{P}$, `PurePath.suffix` $\mathbb{P}$, `path.parse` $\mathbb{N}$). Other syntactic operators compute the relative path between two paths (e.g., `PurePath.relative_to` $\mathbb{P}$, `path.relative` $\mathbb{N}$), common sub-paths (e.g., `os.path.commonpath` $\mathbb{P}$), or normalizations (e.g., `path.normalize` $\mathbb{N}$, `os.path.normpath` $\mathbb{P}$, `Path::components` $\mathbb{R}$). Some libraries also convert between path values and encoded or string representations (e.g., `os.fsencode` $\mathbb{P}$, `Path::to_str` $\mathbb{R}$). Relative paths connect path syntax to ambient process state, where the current working directory is prepended when no base directory is supplied explicitly (e.g., `getcwd` $\mathbb{G}$, `chdir` $\mathbb{G}$, `Path.cwd` $\mathbb{P}$).

File system tree. A path is used to access data in the *file system tree* $\mathbb{T}$. Tree-level operators create (e.g., `link` $\mathbb{G}$, `mkdir` $\mathbb{G}$, `fs.mkdirSync` $\mathbb{N}$), delete (e.g., `unlink` $\mathbb{G}$, `rmdir` $\mathbb{G}$, `Path.unlink` $\mathbb{P}$), rename

Table 1. **Summary of the library survey.** Each family of behaviors is shown with the number of behaviors (#) and the number of distinct APIs in the library which realize at least one of the behaviors (or – if none do): **G** glibc 2.41, **P** Python 3.13, **N** Node.js 24.1, **R** Rust 1.96, Σ all; and the data abstractions on which it operates: **p** path; **t** file tree; **n** node type; **b** block device; **h** character device; **o** FIFO; **k** socket; **s** directory stream; **c** file content; **x** file descriptor; **a** file storage; **m** file metadata; **r** file permissions; **i** file timestamps; **l** soft link; **g** glob pattern; **f** temporary file; **w** current working directory; **e** environment; **q** string.

behavior family	#	G	P	N	R	Σ	abs.
<i>total</i>	128	99	163	121	141	524	
<i>paths</i>	25	3	34	13	39	89	
path decomposition	12	3	19	6	19	47	p
path composition	3	–	5	4	11	20	p t q
path conversion	4	–	3	1	8	12	p q
path computation	5	–	5	1	3	9	p
path normalization	1	–	3	2	2	7	p
<i>tree-level</i>	39	40	62	41	50	193	
file examination	6	4	18	9	18	49	p t n
file creation	4	9	9	4	10	32	p t r
directory traversal	11	10	7	8	5	30	p t n s
file deletion	3	4	8	3	3	18	p t
file copying	2	2	6	2	1	11	p t c
file renaming	2	2	6	1	1	10	p t
path canonicalization	1	2	2	2	2	8	p t
soft link examination	2	2	5	3	5	15	p t n l
special file examination	4	–	–	8	4	12	p t n
soft link creation	1	2	2	1	1	6	p t l
special file creation	3	3	2	–	–	5	p t n b h o k r
<i>file-level</i>	41	47	48	56	65	216	
file permissions	8	13	23	13	21	70	p t r
file timestamps	3	9	10	14	14	47	p t i
file access	9	11	11	15	8	45	p t c x a
file opening	4	8	7	13	16	44	p t c x r
file metadata	3	4	5	10	15	34	p t x m
file synchronization	4	4	3	2	2	11	t c x
file descriptors	5	6	3	–	1	10	s c x
file locking	3	2	–	–	5	7	t c x
file comparison	2	–	–	–	–	2	p t c
<i>extra</i>	23	18	33	19	2	72	
process state	4	5	7	1	2	15	p r w
file system metadata	3	2	4	9	–	15	p t x a
temporary files	4	6	7	1	–	14	p t c x r f g
tree traversal	4	4	6	1	–	11	p t r g
pattern matching	1	1	3	1	–	5	p g
process environment	3	–	4	1	–	5	p t e q
URI manipulation	2	–	2	2	–	4	p q
tree watching	2	–	–	3	–	3	p t

(e.g., rename **G**, Path.rename **P**), and traverse (e.g., opendir **G**, fs.opendirSync **N**) nodes within the tree. The libraries also feature semantic path manipulation operators which normalize paths with respect to the actual tree (e.g., realpath **G**, Path.resolve **P**). Beyond regular files and directories, nodes may take special types—soft links, FIFOs, sockets, and block or character devices—which the libraries create (e.g., symlink **G**, os.mknode **P**) and examine (e.g., Path.readlink **P**, Stats.isSocket **N**). Nodes in the file tree may carry additional data: for instance, *permissions* **r** restricting user access to the node (e.g., chmod **G**, Path.chmod **P**); *timestamps* **i** recording creation, modification, and access events (e.g., utime **G**, fs.utimesSync **N**); and, in the case of regular files, *content* payload **c** in raw bytes (e.g., Path.read_text **P**, fs::read **R**).

Extra. The behaviors under *extra* are heterogeneous: allocation of temporary files **f** (e.g., mktemp **G**); querying file system metadata (e.g., statvfs **G**, fs.statfsSync **N**); glob-based pattern **g** matching on paths (e.g., path.matchesGlob **N**); conversion between paths and URIs (e.g., Path.as_uri **P**); variable expansion (e.g., Path.expanduser **P**); and access to ambient process state such as the current working directory (e.g., getcwd **G**, chdir **G**) or process root (e.g., unix::fs::chroot **R**).

3.2 Survey observations: an API core with enrichments

Through the survey we identify a core subset of behavior families (*paths* and most of *tree-level*) that is expressive enough to describe many programs that interact with the file system, by manipulating paths and using them to access and mutate the file system tree. The remaining behaviors relate to this baseline in one of three ways. Most augment the data at *nodes* within the tree, some enrich the tree *structure* itself, and others are *peripheral* beside the core.

3.2.1 The core. Paths are sequences of components and hence compose via concatenation and factor into *dirname* and *basename*.

$$\begin{aligned}\text{join} &: \text{Path} \times \text{Path} \rightarrow \text{Path} \\ \text{dir, base} &: \text{Path} \rightarrow \text{Path}\end{aligned}$$

Every family of behaviors is mediated by the resolution of a path against the file system tree—the interpretation of a path to reach a location from which file system data is accessed and mutated. This triad is fundamental to nearly all behaviors: the file system is, at its core, the apparatus to manipulate a tree via the resolution of paths.

The tree is a named hierarchy of directories and files, equipped with primitive actions to query, create, delete, and move nodes, and to read through the entries of a directory:

$$\begin{aligned}\text{stat} &: \text{Tree} \times \text{Path} \rightarrow \text{FileType} \\ \text{create, mkdir} &: \text{Tree} \times \text{Path} \rightarrow \text{Tree} \\ \text{unlink, rmdir} &: \text{Tree} \times \text{Path} \rightarrow \text{Tree} \\ \text{rename} &: \text{Tree} \times \text{Path} \times \text{Path} \rightarrow \text{Tree} \\ \text{opendir} &: \text{Tree} \times \text{Path} \rightarrow \text{Directory} \\ \text{readdir} &: \text{Tree} \times \text{Directory} \rightarrow \text{Name} \times \text{Directory}\end{aligned}$$

Following the terminology used in §2, each operator (except for *readdir*) resolves each path to a *location*, representing some (possibly absent) position in the tree.

As seen in §2.2, normalization simplifies the path into a form consisting of only ordinary names, that is, free of *.*, *..*, and the empty component. These are the canonical (simple) paths to locations in the tree. In fact, normalization *is* the mapping from paths to locations, *exposing path resolution as a user-facing operator*.

However, normalization comes in two forms with complementary guarantees. *Syntactic* normalization (e.g., `path.normalize` **N**, `Path::normalize_lexically` **R**) rewrites a path purely as syntax, eliminating non-linear components without consulting the tree; it is thus *total*, but the result need not resolve to the same location nor even at all. *Semantic* normalization (e.g., `realpath` **G**, `Path.resolve` **P**, `fs.realpath` **N**, `fs::canonicalize` **R**) instead resolves the path against the tree and reconstructs the canonical path to the target; it is *partial*—it may fail when the path does not resolve—but faithful, always yielding the canonical path to the same target.

$$\begin{aligned}\text{norm} &: \text{Path} \rightarrow \text{Path} \\ \text{realpath} &: \text{Tree} \times \text{Path} \rightarrow \text{Path}\end{aligned}$$

3.2.2 Enrichments of the core. *Per-node* enrichments extend the core with additional data attached to individual nodes. These include permissions, timestamps, and file content, with families of operators to query and mutate each. Other examples include device, FIFO, and socket nodes, and extended (non-regular) attributes.

Other enrichments are not localizable to individual nodes but alter the effective positional relationships between locations. Soft links, for instance, redirect resolution to follow the linked path before

Name	a	PVar	x, y, r
Component	$q ::= a \mid \epsilon \mid \cdot \mid \dots$	Val	$v ::= b \mid p \mid y$
Path	$p ::= \epsilon \mid p/q$	Expr	$e ::= x \mid v \mid e = e \mid !e \mid e \&\& e \mid e \parallel e$
Bool	$b ::= \text{true} \mid \text{false}$		$\mid \text{join}(e, e) \mid \text{base}(e) \mid \text{dir}(e)$
FileType	$y ::= \text{file} \mid \text{dir}$		$\mid \text{norm}(e)$
Command	$c ::= \text{skip} \mid x := e \mid c \ ; \ c \mid \text{if } e \text{ then } c \text{ else } c \mid \text{while } e \text{ do } c \text{ done}$ $\mid \text{create}(e) \mid \text{mkdir}(e) \mid \text{unlink}(e) \mid \text{rmdir}(e) \mid \text{rename}(e, e)$ $\mid x := \text{stat}(e) \mid x := \text{realpath}(e) \mid x := \text{opendir}(e) \mid r, x, y := \text{readdir}(e)$		

Figure 2. Syntax of WHILE_{FS}.

continuing with the original path. Similarly, mounts, overlays, and chroot each attach another file system at a specific path, so that resolution crossing a mount transparently descends into the grafted tree. Unlike node-level enrichments, these additions affect *where* paths resolve and give the tree a much more complex resolution structure admitting cycles and aliasing among linear paths.

The remaining behaviors sit at the periphery of the core. Some of these express auxiliary conveniences (e.g., glob matching, temporary files, and URIs), while others reflect ambient state relevant to program interactions with the file system, such as file descriptors. The current working directory facilitates the use of relative paths, and certain environment variables, such as HOME, may give paths to certain file system resources.

3.3 The WHILE_{FS} language

The core identified from the survey informs the design of an archetypal programming language that faithfully captures essential file system behaviors and can serve as the basis for further study. Described in figure 2, WHILE_{FS} extends a standard imperative WHILE-style base with paths and file trees. Each construct is motivated by one or more core behaviors from the survey. Commands operate over a program state consisting of a variable store ρ , which maps variables to values, and a file tree τ . Each file system command evaluates arguments, resolves paths, and mutates the file tree.

Paths and expressions. We model paths p as (possibly empty) sequences of components q , each of which is either a name a , the empty component ϵ , the self reference ‘ $\cdot$ ’, or the parent reference ‘ $\dots$ ’. They are composed and decomposed via primitive expressions join, base, dir, and norm, which reflect the core syntactic path behaviors discussed above. Paths can also be compared for syntactic equality via $p_1 = p_2$. Besides paths, other values include booleans and file types file and dir.

Commands and program state. Aside from standard control flow constructs, WHILE_{FS} provides nine file system commands, reflecting the core described above. Here, the file system state over which commands operate is implicit in the program state rather than passed explicitly. Formally, a program state ς is a pair $\langle \rho, \tau \rangle$ with a variable store ρ and a file tree τ .

File trees. As in figure 3, we model file trees inductively. A file tree τ is a file or a (dir δ) with the entries δ , a finite mapping of names to child trees. To facilitate local updates on a tree, we adopt standard machinery using one-holed contexts [1, 41]. In this case, a context C is a file tree with a single hole $\square$ at a particular location, and filling the hole with a sub-tree τ recovers a tree $C[\tau]$. In essence, the context and focus together represent a tree with some sub-tree picked out. A context may also be filled with $\perp$ to erase the hole without planting a sub-tree. Note that in the definition of $C[f]$, we write $\delta[a \mapsto \tau]$ to denote an update to the map.

FileTree	τ	$::=$	file dirc δ	FTCtx	C	$::=$	$\square$ dirc $\delta[a \mapsto C]$
FileEntries	δ	$\in$	Name $\rightarrow_{\text{fin}}$ FileTree	FTFocus	f	$::=$	τ $\perp$
$C[f] \in \text{FileTree}$ Fill the hole in C with f .				resolve $p \tau \in \text{FTCtx} \times \text{FTFocus}$ Resolve p in τ .			
$\square[\tau] \triangleq \tau$				resolve $p \tau \triangleq \langle \text{ctx } z, \text{focus } z \rangle$			
$(\text{dirc } \delta[a \mapsto \square])[\perp] \triangleq \text{dirc } \delta$				iff traverse p (zip τ) = z			
$(\text{dirc } \delta[a \mapsto C])[f] \triangleq \text{dirc } \delta[a \mapsto C[f]]$							

Figure 3. File trees, one-holed contexts, and path resolution.

Successful resolution of a path returns a one-holed context C for the target’s location and the *focus* f —the sub-tree at that location, or $\perp$ if there is none. Since $\dots$ within a path may dictate an upward traversal, to model path resolution we use *zipper*s à la Huet [27], which encode one-holed contexts in an inverted fashion, maintaining upward structure from the hole to facilitate movement to both the parent and children. In this setting, a zipper z is a stack of *frames* encoding a context in layers down to the root, paired with the focus extracted from the hole. $\text{resolve } p \tau$ constructs a zipper at the root of τ , then traverses through the tree along each component, first checking that the focus is on a directory, then going down to a child for names, going up to the parent for $\dots$, and staying put for $\cdot$ and ϵ (see Appendix C for exact definitions). If the target location is successfully reached, we reify the zipper back into a context ($\text{ctx } z$) and extract the focus ($\text{focus } z$).

Small-step semantics. Figure 4 gives the operational semantics for file system-related operators and commands. We write $e \Downarrow^p v$ for evaluation of expressions under the variable store. join , base , and dir are straightforward, and norm proceeds by folding over the path and popping off the last name for each ‘ $\dots$ ’. During execution, values may additionally include directory entry name sets Δ . For a command c , we write $(c, \varsigma) \rightarrow^1 (c', \varsigma')$ and $(c, \varsigma) \rightarrow^* (c', \varsigma')$ for a single step and many steps, respectively.

The semantics for file system commands employs resolve for path resolution and context-filling for mutation. For instance, $\text{create}(e)$ resolves the path (evaluated from e) to a context with an absent focus, fills the hole with a new file node, and returns true to r . mkdir , unlink , rmdir , and stat follow similarly with different focus checks and fills. $\text{rename}(e_1, e_2)$ resolves the source path to a non- $\perp$ (i.e., file or dirc) focus, erases the hole by filling with $\perp$, resolves the destination path to a $\perp$ focus, and fills that hole with the source’s subtree. $x \coloneqq \text{realpath}(e)$ resolves the path then reifies the target context C into a canonical path. Finally, $x \coloneqq \text{opendir}(e)$ sets x to the set of entry names $\text{dom } \delta$, and $r, x, y \coloneqq \text{readdir}(e)$ selects a name out of the set if it is not empty.

4 TWE: A THEORY FOR FILE SYSTEM REASONING

This section develops a theory for the core operations distilled in WHILE_{FS} (§3). The theory, introduced in §4.1, draws upon the observations identified from the survey to enable reasoning over interactions between paths and the walks they induce. Central to the theory are *tree-walking word equations*, with further ingredients to support path normalization and directory reasoning. To support mutations to the tree, the theory supports functional updates similarly to standard theories of arrays. We call this language TWE. §4.2 then demonstrates its adequacy for the WHILE_{FS} core through a symbolic semantics which emits constraints in the theory. We establish soundness and completeness at each step of this symbolic semantics with respect to concrete executions, and show how a symbolic trace captures the failing execution of rmr in listing 2 from §2. Note that we do not develop a full program logic nor claim whole-program specification; TWE is an underlying layer

$\text{join } p_1 p_2, \text{base } p, \text{dir } p, \text{norm } p \in \text{Path}$

$$\begin{array}{ll}
 \text{join } p \varepsilon \triangleq p & \text{norm } \varepsilon \triangleq \varepsilon \\
 \text{join } p_1 (p_2/q) \triangleq (\text{join } p_1 p_2)/q & \text{norm } (p/a) \triangleq (\text{norm } p)/a \\
 \text{base } \varepsilon \triangleq \varepsilon & \text{norm } (p/\varepsilon) \triangleq \text{norm } p \\
 \text{base } (_/q) \triangleq q & \text{norm } (p/\cdot) \triangleq \text{norm } p \\
 \text{dir } \varepsilon \triangleq \varepsilon & \text{norm } (p/\cdot) \triangleq \text{dir } (\text{norm } p) \\
 \text{dir } (p/\cdot) \triangleq p &
 \end{array}$$

$e \Downarrow^p v$ Under p , e evaluates to v . $(c, \zeta) \longrightarrow^1 (c', \zeta')$ Under ζ , c steps to c' with new state ζ' .

Val $v ::= \dots \mid \Delta$
Store $p \in \text{PVar} \rightarrow \text{Val}$

Directory $\Delta \in \mathcal{P}_{\text{fin}}(\text{Name})$
State $\zeta ::= \langle p, \tau \rangle$

$\frac{\text{EVALVAR} \quad \frac{p x = v}{x \Downarrow^p v}}{\text{EVALJOIN} \quad \frac{e_1 \Downarrow^p p_1 \quad e_2 \Downarrow^p p_2}{\text{join}(e_1, e_2) \Downarrow^p \text{join } p_1 p_2}}$	$\frac{\text{EVALBASE} \quad \frac{e \Downarrow^p p}{\text{base}(e) \Downarrow^p \text{base } p}}{\text{EVALDIR} \quad \frac{e \Downarrow^p p}{\text{dir}(e) \Downarrow^p \text{dir } p}}$	$\frac{\text{EVALNORM} \quad \frac{e \Downarrow^p p}{\text{norm}(e) \Downarrow^p \text{norm } p}}$
$\frac{\text{STEPCREATE} \quad \frac{e \Downarrow^p p}{\text{res } p \tau = \langle C, \perp \rangle}}{\longrightarrow^1 (\text{create}(e), \langle p, \tau \rangle)} \longrightarrow^1 (\text{skip}, \langle p, C[\text{file}] \rangle)$	$\frac{\text{STEPMKDIR} \quad \frac{e \Downarrow^p p}{\text{res } p \tau = \langle C, \perp \rangle}}{\longrightarrow^1 (\text{mkdir}(e), \langle p, \tau \rangle)} \longrightarrow^1 (\text{skip}, \langle p, C[\text{dir } \emptyset] \rangle)$	$\frac{\text{STEPUNLINK} \quad \frac{e \Downarrow^p p}{\text{res } p \tau = \langle C, \text{file} \rangle}}{\longrightarrow^1 (\text{unlink}(e), \langle p, \tau \rangle)} \longrightarrow^1 (\text{skip}, \langle p, C[\perp] \rangle)$
$\frac{\text{STEPRENAME} \quad \frac{e_1 \Downarrow^p p_1 \quad \text{res } p_1 \tau = \langle C_1, \tau_1 \rangle}{e_2 \Downarrow^p p_2 \quad \text{res } p_2 C_1[\perp] = \langle C_2, \perp \rangle}}{\longrightarrow^1 (\text{rename}(e_1, e_2), \langle p, \tau \rangle)} \longrightarrow^1 (\text{skip}, \langle p, C_2[\tau_1] \rangle)$	$\frac{\text{STEPSTAT} \quad \frac{e \Downarrow^p p}{\text{res } p \tau = \langle C, \tau' \rangle}}{\longrightarrow^1 ((x := \text{stat}(e)), \langle p, \tau \rangle)} \longrightarrow^1 (\text{skip}, \langle p[x \mapsto \text{stat } \tau'], \tau \rangle)$	$\frac{\text{STEPREALPATH} \quad \frac{e \Downarrow^p p}{\text{res } p \tau = \langle C, _ \rangle}}{\longrightarrow^1 (x := \text{realpath}(e), \langle p, \tau \rangle)} \longrightarrow^1 (\text{skip}, \langle p[x \mapsto \text{path } C], \tau \rangle)$
$\frac{\text{STEPOPENDIR} \quad \frac{e \Downarrow^p p}{\text{res } p \tau = \langle C, \text{dir } \delta \rangle}}{\longrightarrow^1 ((x := \text{opendir}(e)), \langle p, \tau \rangle)} \longrightarrow^1 (\text{skip}, \langle p[x \mapsto \text{dom } \delta], \tau \rangle)$	$\frac{\text{STEPREADDIRNIL} \quad \frac{e \Downarrow^p \Delta \quad \Delta = \emptyset}}{\longrightarrow^1 ((r, x, y := \text{readdir}(e)), \langle p, \tau \rangle)} \longrightarrow^1 (\text{skip}, \langle p[r \mapsto \text{false}], x \mapsto \Delta, \tau \rangle)$	$\frac{\text{STEPREADDIRCONS} \quad \frac{e \Downarrow^p \Delta \quad a \in \Delta \quad \Delta' = \Delta \setminus \{a\}}{p' = p[r \mapsto \text{true}][x \mapsto \Delta'][y \mapsto a]}}{\longrightarrow^1 ((r, x, y := \text{readdir}(e)), \langle p, \tau \rangle)} \longrightarrow^1 (\text{skip}, \langle p', \tau \rangle)$

Figure 4. **Operational semantics for WHILEFS.** We elide the rules for standard constructs (see Appendix E.2).

capturing the constraint problem of path and file tree behavior. Even so, the theory turns out to be quite hard, with decidability at least as difficult as longstanding open problems as demonstrated in §5. We identify decidable fragments in §6.

4.1 A theory of tree-walking word equations with updates

The theory (figure 5) consists of quantifier-free propositional formulae constructed over predicates on paths. First, we recall the standard theory of word equations, using them to reason syntactically about paths. Then, to model their semantics, we introduce *walks* over trees with functional updates.

4.1.1 Syntactic reasoning with word equations. Recall that a path p is a finite sequence of an infinite alphabet of components. In a program, the value of a path may not be known or fixed, so we also admit *path variables* X standing for unknown paths; in general, a *word* U is constructed by concatenating components and variables (figure 5), and these describe path expressions. For instance, the variable-free word `app · config` corresponds to the path `/app/config`, and `app · X` stands for paths of the form `/app/X`. The empty word ϵ corresponds to the root path `/`, which has no components. Since operators manipulating paths (e.g., `base`) cannot be evaluated directly

Name	$A ::= X_A \mid a$	Tree	$T ::= V \mid \text{file} \mid \text{dir}_{\emptyset} \mid \perp$
Comp	$B ::= X_B \mid \epsilon \mid \cdot \mid \cdot \mid A$		$\mid T[L] \mid T[L \mapsto T]$
Path	$P, Q ::= X \mid \epsilon \mid B \mid P \cdot Q$	Shape	$H ::= \perp \mid \text{file} \mid \text{dir}$
	$\mid L \mid \text{norm } P$	Prop	$\Phi ::= \dots \mid P = Q \mid \text{IsLin } P$
Names	$S ::= R \mid \emptyset \mid S \cup A \mid \text{entries } T$		$\mid P \rightarrow_T L \mid T \in H \mid S = S' \mid A \in S$
Subst	$\sigma \in \text{Var} \rightarrow \text{Path}$	Forest	$\theta \in \text{TreeVar} \rightarrow \text{FileTree}$

$\llbracket P \rrbracket_{\sigma, \theta} \in \text{Path}$		$\llbracket S \rrbracket_{\sigma, \theta} = \mathcal{P}_{\text{fin}}(\text{Name})$	
$\llbracket X \rrbracket_{\sigma, \theta} \triangleq \sigma X$		$\llbracket R \rrbracket_{\sigma, \theta} \triangleq \sigma R$	σR is a set
$\llbracket \epsilon \rrbracket_{\sigma, \theta} \triangleq \epsilon$		$\llbracket \emptyset \rrbracket_{\sigma, \theta} \triangleq \emptyset$	
$\llbracket B \rrbracket_{\sigma, \theta} \triangleq \dots$		$\llbracket S \cup A \rrbracket_{\sigma, \theta} \triangleq \llbracket S \rrbracket_{\sigma, \theta} \cup \llbracket A \rrbracket_{\sigma, \theta}$	
$\llbracket P \cdot Q \rrbracket_{\sigma, \theta} \triangleq \text{join } \llbracket P \rrbracket_{\sigma, \theta} \llbracket Q \rrbracket_{\sigma, \theta}$		$\llbracket \text{entries } T \rrbracket_{\sigma, \theta} \triangleq \text{dom } \delta$	$\llbracket T \rrbracket_{\sigma, \theta} = \text{dir } \delta$
$\llbracket L \rrbracket_{\sigma, \theta} \triangleq \sigma L$			
$\llbracket \text{norm } P \rrbracket_{\sigma, \theta} \triangleq \text{norm } \llbracket P \rrbracket_{\sigma, \theta}$			

$\llbracket T \rrbracket_{\sigma, \theta} \in \text{FTFocus}$		$\text{shape } f \in \text{Shape}$	
$\llbracket V \rrbracket_{\sigma, \theta} \triangleq \theta V$		$\text{shape } \perp \triangleq \perp$	
$\llbracket \text{file} \rrbracket_{\sigma, \theta} \triangleq \text{file}$		$\text{shape } \text{file} \triangleq \text{file}$	
$\llbracket \text{dir}_{\emptyset} \rrbracket_{\sigma, \theta} \triangleq \text{dir } \emptyset$		$\text{shape } (\text{dir } \delta) \triangleq \text{dir } \delta$	
$\llbracket \perp \rrbracket_{\sigma, \theta} \triangleq \perp$			
$\llbracket T[L] \rrbracket_{\sigma, \theta} \triangleq f$			
$\text{resolve } \llbracket L \rrbracket_{\sigma, \theta} \llbracket T \rrbracket_{\sigma, \theta} = \langle C, f \rangle$			
$\llbracket T[L \mapsto T'] \rrbracket_{\sigma, \theta} \triangleq C[\llbracket T' \rrbracket_{\sigma, \theta}]$			
$\text{resolve } \llbracket L \rrbracket_{\sigma, \theta} \llbracket T \rrbracket_{\sigma, \theta} = \langle C, _ \rangle$			

$\langle \sigma, \theta \rangle \models \Phi$			
$\langle \sigma, \theta \rangle \models P = Q$	iff	$\llbracket P \rrbracket_{\sigma, \theta} = \llbracket Q \rrbracket_{\sigma, \theta}$	
$\langle \sigma, \theta \rangle \models \text{IsLin } P$	iff	$\llbracket P \rrbracket_{\sigma, \theta} \text{ lin}$	
$\langle \sigma, \theta \rangle \models P \rightarrow_T L$	iff	$\text{resolve } \llbracket P \rrbracket_{\sigma, \theta} \llbracket T \rrbracket_{\sigma, \theta} = \langle C, _ \rangle;$	
		$\text{path } C = \llbracket L \rrbracket_{\sigma, \theta}$	
$\langle \sigma, \theta \rangle \models T \in H$	iff	$\text{shape } \llbracket T \rrbracket_{\sigma, \theta} = H$	
$\langle \sigma, \theta \rangle \models S = S'$	iff	$\llbracket S \rrbracket_{\sigma, \theta} = \llbracket S' \rrbracket_{\sigma, \theta}$	
$\langle \sigma, \theta \rangle \models A \in S$	iff	$\llbracket A \rrbracket_{\sigma, \theta} \in \llbracket S \rrbracket_{\sigma, \theta}$	

Figure 5. The theory of tree-walking word equations with updates.

when the values of their operands are unknown, we reason about the shape of words via *equations* between words: $U = V$.

This theory of *word equations*, which we now briefly recall, is well-studied in the literature [29, 40, 46]. A *substitution* σ maps variables and, by extension, words to sequences of letters from a finite or infinite alphabet; we write σX and $\llbracket U \rrbracket_{\sigma}$, respectively. A substitution is a *solution* satisfying the equation $U = V$ if and only if $\llbracket U \rrbracket_{\sigma} = \llbracket V \rrbracket_{\sigma}$. An instance of the word equation problem asks whether a system of equations has a satisfying substitution. In our case, since words represent paths in our setting, we will write P and Q in lieu of U and V .

The join operation in WHILE_{FS} corresponds exactly to concatenation of words. Moreover, the behavior of several path operators can be specified using word equations. The result of an assignment $X_B := \text{base}(P)$ can be expressed as a disjunction in two cases, depending on if p is the root path or not: (1) if $P = \epsilon$, then $X_B = \epsilon$; or (2) if $P \neq \epsilon$, then $P = Y \cdot X_B$, where Y is a fresh path variable, and X_B is a fresh *component* variable. Here, we permit certain variables X_B to denote single components (instead of sequences of components) and X_A to denote strictly names (and not ϵ , $\cdot$, or $\dots$). Note that this sort distinction does not hamper the theory's decidability [31, Theorem 3]. Similarly, for $X := \text{dir}(P)$, we may write, for some fresh path and component variables X and Y_B , the disjunction $((P = \epsilon) \wedge (X = \epsilon)) \vee ((P \neq \epsilon) \wedge (P = X \cdot Y_B))$.

4.1.2 Tree reasoning with walks and shapes. When resolved against the file tree, a path induces a walk through the tree which, if successful, resolves to a location in the tree. In the semantics of WHILE_{FS} , locations were represented by one-holed contexts C (§3.3). In TWE, we distinguish locations L as canonical paths, equivalent to path C , and permit treatment of them as paths. To model path resolution, a *walk assertion* $P \rightarrow_T L$ asserts that P resolves to the location L in the tree T . The walk assertion does not claim anything about the node at the *target* location—which may be a file, directory, or absent—only that the path resolves successfully to the location. Recall that every *intermediate* location must, however, be a directory with the necessary children. A separate *shape assertion* $T[L] \in H$ states that the location has the shape H , either file, dirc, or $\perp$ for absent. A tree term T is abstract (variables V), a file (file), an empty directory ($\text{dirc}_{\emptyset}$), absent ($\perp$), or the sub-tree at a location in another tree $T'[L]$.

Already we can partially characterize a wide range of programs which handle paths and the file tree. For instance, we can write a precondition for a successful invocation $\text{unlink}(P)$ with a formula demanding that P resolves to a location L on the current tree T and the location is occupied by a file: $(P \rightarrow_T L) \wedge (T[L] \in \text{file})$. As pairing walks and shape assertions is common, we write $(P \rightarrow_T L \in H)$ to mean $(P \rightarrow_T L) \wedge (T[L] \in H)$.

Whereas solutions for word equations range over substitutions, tree walks additionally quantify existentially over file trees. Precisely, as in figure 5, a model consists of a substitution σ and a *tree assignment* θ mapping each tree variable V to a τ . The substitution also maps each location symbol L to a linear path describing a location. The model satisfies a walk assertion $P \rightarrow_T L$ if and only if the substituted path $\llbracket P \rrbracket_{\sigma, \theta}$ resolves successfully to the assigned location σL within the assigned tree $\llbracket T \rrbracket_{\sigma, \theta}$. Note that tree terms evaluate to *foci* rather than directly to file trees, since a tree term may stand for an absent focus. For simplicity, at the model level we reuse σ for set values.

4.1.3 Functional tree updates. To model mutations to file trees, it is necessary to support updates to trees. To this end, we adopt functional updates similarly to the theory of arrays à la McCarthy [42], generalizing the select/store discipline into tree-shaped versions. A selection $T[L]$ gives the sub-tree at L in T , and an update $T[L \mapsto T']$ gives a new tree from T by replacing the sub-tree at L with T' . Now it is possible to specify, for instance, the deletion of a file via $\text{unlink}(P)$ with the update $T[L \mapsto \perp]$.

4.1.4 Tree entries. To allow reasoning over the entries of a directory, entries T denotes a set S consisting of exactly the entry names at the root of the directory T . A set is abstract (variables R), empty ($\emptyset$), or the union with a name ($S \cup A$). The theory allows testing for set equality $S = S'$ and membership $A \in S$. Given this, we can write a post-condition for $R := \text{opendir}(P)$, which resolves P and sets R to the entry set: $(P \rightarrow_T L \in \text{dirc}) \wedge (R = \text{entries } T[L])$.

4.1.5 Normalization. Observe that with walks and equalities, it is possible to specify semantic normalization. For $Q := \text{realpath}(P)$, we have the post-condition $(P \rightarrow_T L) \wedge (L = Q)$. However, *syntactic* normalization cannot be specified precisely by word equations, nor is it regular. We therefore admit normalization as a separate word operator $\text{norm } P$. Then, for $Q := \text{norm}(P)$, we have the post-condition $Q = \text{norm } P$.

4.2 Symbolic semantics for WHILE_{FS}

To demonstrate that the logic of tree-walking word equations is adequate to reason about the file system core and WHILE_{FS} , we develop symbolic operational semantics (figure 6) which simulate the concrete semantics given in §3.3. The symbolic semantics generate conditions in the theory of tree-walking word equations and are sound and complete with respect to concrete execution steps

$$\begin{array}{c}
 \widehat{\text{Val}} \quad \hat{v} ::= \Phi \mid P \mid T \mid S \quad \widehat{\text{PC}} \quad \hat{\pi} \in \text{Prop} \\
 \widehat{\text{Store}} \quad \hat{\rho} \in \text{PVar} \rightarrow_{\text{fin}} \widehat{\text{Val}} \quad \widehat{\text{State}} \quad \hat{\xi} ::= \langle \hat{\rho}, T, \hat{\pi} \rangle
 \end{array}$$

$e \Downarrow_{\Phi}^{\hat{\rho}} \hat{v}$ Under $\hat{\rho}$, e evaluates to $\hat{v}$, generating Φ .

$(c, \hat{\xi}) \rightarrow^1 (c', \hat{\xi}')$ Under $\hat{\xi}$, c steps to c' with new state $\hat{\xi}'$.

$ \begin{array}{c} \text{SEVAEQFILETYPE} \\ \frac{e_1 \Downarrow_{\Phi_1}^{\hat{\rho}} T_1 \quad e_2 \Downarrow_{\Phi_2}^{\hat{\rho}} T_2 \quad \Phi' = \left(\bigvee_H T_1 \in H \wedge T_2 \in H \right)}{e_1 = e_2 \Downarrow_{\Phi_1 \wedge \Phi_2}^{\hat{\rho}} \Phi'} \end{array} $	$ \begin{array}{c} \text{SEVAEQPATH} \\ \frac{e_1 \Downarrow_{\Phi_1}^{\hat{\rho}} P_1 \quad e_2 \Downarrow_{\Phi_2}^{\hat{\rho}} P_2}{e_1 = e_2 \Downarrow_{\Phi_1 \wedge \Phi_2}^{\hat{\rho}} P_1 = P_2} \end{array} $	$ \begin{array}{c} \text{SEVALJOIN} \\ \frac{e_1 \Downarrow_{\Phi_1}^{\hat{\rho}} P_1 \quad e_2 \Downarrow_{\Phi_2}^{\hat{\rho}} P_2}{\text{join}(e_1, e_2) \Downarrow_{\Phi_1 \wedge \Phi_2}^{\hat{\rho}} P_1 \cdot P_2} \end{array} $
$ \begin{array}{c} \text{SEVALBASE} \\ \frac{e \Downarrow_{\Phi}^{\hat{\rho}} P \quad \Phi' = (P = X \cdot Y_B) \vee (P = \epsilon \wedge Y_B = \epsilon)}{\text{base}(e) \Downarrow_{\Phi \wedge \Phi'}^{\hat{\rho}} Y_B} \end{array} $	$ \begin{array}{c} \text{SEVALDIR} \\ \frac{e \Downarrow_{\Phi}^{\hat{\rho}} P \quad \Phi' = (P = X \cdot Y_B) \vee (P = \epsilon \wedge X = \epsilon)}{\text{dir}(e) \Downarrow_{\Phi \wedge \Phi'}^{\hat{\rho}} X} \end{array} $	$ \begin{array}{c} \text{SEVALNORM} \\ \frac{e \Downarrow_{\Phi}^{\hat{\rho}} P}{\text{norm}(e) \Downarrow_{\Phi}^{\hat{\rho}} \text{norm } P} \end{array} $
$ \begin{array}{c} \text{SSTEPCREATE} \\ \frac{e \Downarrow_{\Phi}^{\hat{\rho}} P \quad \text{SAT}(\hat{\pi}') \quad \hat{\pi}' = \hat{\pi} \wedge \Phi \wedge P \rightarrow_T L \in \perp}{(\text{create}(e), \langle \hat{\rho}, T, \hat{\pi}' \rangle) \rightarrow^1 (\text{skip}, \langle \hat{\rho}, T[L \mapsto \text{file}], \hat{\pi}' \rangle)} \end{array} $	$ \begin{array}{c} \text{SSTEPMKDIR} \\ \frac{e \Downarrow_{\Phi}^{\hat{\rho}} P \quad \text{SAT}(\hat{\pi}') \quad \hat{\pi}' = \hat{\pi} \wedge \Phi \wedge P \rightarrow_T L \in \perp}{(\text{mkdir}(e), \langle \hat{\rho}, T, \hat{\pi}' \rangle) \rightarrow^1 (\text{skip}, \langle \hat{\rho}, T[L \mapsto \text{dir}_{\emptyset}], \hat{\pi}' \rangle)} \end{array} $	$ \begin{array}{c} \text{SSTEPUNLINK} \\ \frac{e \Downarrow_{\Phi}^{\hat{\rho}} P \quad \text{SAT}(\hat{\pi}') \quad \hat{\pi}' = \hat{\pi} \wedge \Phi \wedge P \rightarrow_T L \in \text{file}}{(\text{unlink}(e), \langle \hat{\rho}, T, \hat{\pi}' \rangle) \rightarrow^1 (\text{skip}, \langle \hat{\rho}, T[L \mapsto \perp], \hat{\pi}' \rangle)} \end{array} $
$ \begin{array}{c} \text{SSTEPRMDIR} \\ \frac{e \Downarrow_{\Phi}^{\hat{\rho}} P \quad \hat{\pi}' = \hat{\pi} \wedge \Phi \wedge P \rightarrow_T L \in \text{dir} \wedge \text{entries } T[L] = \emptyset \quad \text{SAT}(\hat{\pi}')}{(\text{rmdir}(e), \langle \hat{\rho}, T, \hat{\pi}' \rangle) \rightarrow^1 (\text{skip}, \langle \hat{\rho}, T[L \mapsto \perp], \hat{\pi}' \rangle)} \end{array} $	$ \begin{array}{c} \text{SSTEPRENAME} \\ \frac{e_1 \Downarrow_{\Phi_1}^{\hat{\rho}} P_1 \quad e_2 \Downarrow_{\Phi_2}^{\hat{\rho}} P_2 \quad \text{SAT}(\hat{\pi}') \quad \hat{\pi}' = \hat{\pi} \wedge \Phi_1 \wedge \Phi_2 \wedge P_1 \rightarrow_T L_1 \notin \perp \wedge P_2 \rightarrow_T [L_1 \mapsto \perp] L_2 \in \perp}{(\text{rename}(e_1, e_2), \langle \hat{\rho}, T, \hat{\pi}' \rangle) \rightarrow^1 (\text{skip}, \langle \hat{\rho}, T[L_1 \mapsto \perp][L_2 \mapsto T[L_1]], \hat{\pi}' \rangle)} \end{array} $	
$ \begin{array}{c} \text{SSTEPSTAT} \\ \frac{e \Downarrow_{\Phi}^{\hat{\rho}} P \quad \hat{\pi}' = \hat{\pi} \wedge \Phi \wedge P \rightarrow_T L \notin \perp \quad \text{SAT}(\hat{\pi}')}{(x = \text{stat}(e), \langle \hat{\rho}, T, \hat{\pi}' \rangle) \rightarrow^1 (\text{skip}, \langle \hat{\rho}[x \mapsto T[L]], T, \hat{\pi}' \rangle)} \end{array} $	$ \begin{array}{c} \text{SSTEPREALPATH} \\ \frac{e \Downarrow_{\Phi}^{\hat{\rho}} P \quad \hat{\pi}' = \hat{\pi} \wedge \Phi \wedge P \rightarrow_T L \notin \perp \quad \text{SAT}(\hat{\pi}')}{(x = \text{realpath}(e), \langle \hat{\rho}, T, \hat{\pi}' \rangle) \rightarrow^1 (\text{skip}, \langle \hat{\rho}[x \mapsto L], T, \hat{\pi}' \rangle)} \end{array} $	$ \begin{array}{c} \text{SSTEPOPENDIR} \\ \frac{e \Downarrow_{\Phi}^{\hat{\rho}} P \quad \hat{\pi}' = \hat{\pi} \wedge \Phi \wedge P \rightarrow_T L \in \text{dir} \wedge R = \text{entries } T[L] \quad \text{SAT}(\hat{\pi}')}{(x = \text{opendir}(e), \langle \hat{\rho}, T, \hat{\pi}' \rangle) \rightarrow^1 (\text{skip}, \langle \hat{\rho}[x \mapsto R], T, \hat{\pi}' \rangle)} \end{array} $
$ \begin{array}{c} \text{SSTEPREADDIRNIL} \\ \frac{e \Downarrow_{\Phi}^{\hat{\rho}} S \quad \hat{\pi}' = \hat{\pi} \wedge \Phi \wedge S = \emptyset \quad \text{SAT}(\hat{\pi}')}{(r, x, y = \text{readdir}(e), \langle \hat{\rho}, T, \hat{\pi}' \rangle) \rightarrow^1 (\text{skip}, \langle \hat{\rho}[r \mapsto \perp], T, \hat{\pi}' \rangle)} \end{array} $	$ \begin{array}{c} \text{SSTEPREADDIRCONS} \\ \frac{e \Downarrow_{\Phi}^{\hat{\rho}} S \quad \hat{\pi}' = \hat{\pi} \wedge \Phi \wedge (S = R \cup X_A) \wedge (X_A \notin R) \quad \text{SAT}(\hat{\pi}')}{(r, x, y = \text{readdir}(e), \langle \hat{\rho}, T, \hat{\pi}' \rangle) \rightarrow^1 (\text{skip}, \langle \hat{\rho}[r \mapsto T][x \mapsto R][y \mapsto X_A], T, \hat{\pi}' \rangle)} \end{array} $	

 Figure 6. Symbolic operational semantics for **WHILE_{FS}**. See Appendix G.3 for full rules.

(theorem 1). Using these semantics, we demonstrate the failing execution of **rmr** of listing 2 from §2 in a symbolic trace.

4.2.1 Program state and judgments. Recall that each program state in concrete executions consists of the variable store ρ and the file tree τ . In the symbolic semantics, we maintain a symbolic variable store $\hat{\rho}$ in the standard way, and TWE assertions accumulated in the path condition $\hat{\pi}$ track the space of possible file trees. Since the tree undergoes mutations, we adopt functional updates via the tree-shaped select and store operators introduced above and track the tree term T corresponding to the state of the file tree. The symbolic program state $\hat{\xi}$ is therefore a triple $\langle \hat{\rho}, T, \hat{\pi} \rangle$. Mirroring the

concrete semantics, we write $(c, \hat{\varsigma}) \longrightarrow^1 (c', \hat{\varsigma}')$ for a single symbolic step and $(c, \hat{\varsigma}) \longrightarrow^* (c', \hat{\varsigma}')$ for many.

Consider the `SSTEPCREATE` rule from figure 6. First, we symbolically evaluate the argument e into a path term P alongside additional side constraints Φ to be accumulated into the path condition. Following the example for unlink given above, for create to succeed, the path must resolve to an absent location ($P \rightarrow_V L \in \perp$). The step may be taken if the pre-condition is satisfiable. The mutation is then modeled by an update $T[L \mapsto \text{file}]$. The rules for `mkdir` and `stat` follow similarly, writing $P \rightarrow_T L \notin H$ as shorthand for $(P \rightarrow_T L) \wedge (T[L] \notin H)$. `rename` combines tree selection and updates to detach the sub-tree at the source location and graft it on at the target. Note that moving the sub-tree into itself is disallowed since the pre-condition demands that P_2 resolves on an updated $T[L_1 \mapsto \perp]$. `opendir` returns entries T as mentioned above, and `readdir` pops from the set S if it is non-empty.

4.2.2 Soundness and completeness. The symbolic operational semantics are sound and complete with respect to concrete executions. Soundness guarantees that every symbolic trace denotes only real executions; conversely, all real executions are captured by symbolic ones.

THEOREM 1 (Soundness and completeness of symbolic semantics).

Soundness. If $(c, \langle \hat{\rho}, T, \hat{\pi} \rangle) \longrightarrow^* (c', \langle \hat{\rho}', T', \hat{\pi}' \rangle)$, then for every $\langle \sigma, \theta \rangle \models \hat{\pi}'$, it is the case that $(c, \llbracket \langle \hat{\rho}, T, \hat{\pi} \rangle \rrbracket_{\sigma, \theta}) \longrightarrow^* (c', \llbracket \langle \hat{\rho}', T', \hat{\pi}' \rangle \rrbracket_{\sigma, \theta})$.

Completeness. If $\langle \sigma, \theta \rangle \models \hat{\pi}$, and $\varsigma = \llbracket \langle \hat{\rho}, T, \hat{\pi} \rangle \rrbracket_{\sigma, \theta}$, and $(c, \varsigma) \longrightarrow^* (c', \varsigma')$, then $(c, \langle \hat{\rho}, T, \hat{\pi} \rangle) \longrightarrow^* (c', \langle \hat{\rho}', T', \hat{\pi}' \rangle)$, and there exists $\langle \sigma', \theta' \rangle$ such that $\langle \sigma', \theta' \rangle \models \hat{\pi}'$, and $\varsigma = \llbracket \langle \hat{\rho}, T, \hat{\pi} \rangle \rrbracket_{\sigma', \theta'}$, and $\varsigma' = \llbracket \langle \hat{\rho}', T', \hat{\pi}' \rangle \rrbracket_{\sigma', \theta'}$.

PROOF. Induction over rules in figure 6; the full rules are given in Appendix G.3. $\square$

These sound and complete symbolic semantics form the basis for bug-finding in `WHILEFS` programs. Concretely, define rules corresponding to those in figure 6 which step to an error if the command *could fail* by asking if the pre-condition *could be unsatisfied*, i.e., if the *negation* of the pre-condition is satisfiable. For instance, `SSTEPSTATFAIL`, corresponding to `SSTEPSTAT`, checks if the path P could fail to resolve or resolve to an absent location:

$$\begin{array}{c} \text{SSTEPSTAT} \\ \frac{e \Downarrow_{\Phi}^{\hat{\rho}} P \quad \hat{\pi}' = \hat{\pi} \wedge \Phi \wedge P \rightarrow_T L \notin \perp \quad \text{SAT}(\hat{\pi}')}{(x := \text{stat}(e), \langle \hat{\rho}, T, \hat{\pi} \rangle) \longrightarrow^1 (\text{skip}, \langle \hat{\rho}[x \mapsto T[L]], T, \hat{\pi}' \rangle)} \end{array} \quad \begin{array}{c} \text{SSTEPSTATFAIL} \\ \frac{e \Downarrow_{\Phi}^{\hat{\rho}} P \quad \text{SAT}(\hat{\pi} \wedge \neg (P \rightarrow_T L \notin \perp))}{(x := \text{stat}(e), \langle \hat{\rho}, T, \hat{\pi} \rangle) \longrightarrow^1 (\text{error}, \langle \hat{\rho}, T, \hat{\pi} \rangle)} \end{array}$$

Bug-finding involves finding symbolic traces which step to error, and theorem 1 ensures that such buggy traces never exhibit false positives, and all true bugs are exhibited by some trace.

Now, recall from listing 2 the failing execution of `rmr(/app/config/..)` on the tree in figure 1, where the directory at `/app/config/` was expected to be a directory after it had been deleted. A symbolic trace can witness this bug in an execution from an abstract initial state $\langle \emptyset, V, \top \rangle$ with abstract path input X . In particular, after `rmkdir( $X \cdot X_A$ )`, where X_A is the first entry name of the directory at X , we have that $X \cdot X_A \rightarrow_V L$ with the tree $V[L \mapsto \perp]$. Then, upon trying to resolve the second entry via `stat( $X \cdot X_A'$ )`, the pre-condition includes $X \cdot X_A' \rightarrow_{V[L \mapsto \perp]} L'$. Observe that $X_A \neq X_A'$ hence $L \parallel L'$, so it is not possible for the target L' of this walk to be within the $\perp$ sub-tree at L . However, it is possible that the pre-condition $X \cdot X_A' \rightarrow_{V[L \mapsto \perp]} L'$ be *unsatisfied*, in particular

if X traverses L , i.e., if $X = Y \cdot Z$ and $Y \rightarrow_{V[L \mapsto \perp]} L$. This triggers `SSTEPSTATFAIL`. Taking $\sigma X = \texttt{/app/config/..}$ and $\theta V = \text{figure 1}$, we recover exactly the scenario in listing 2.

5 WE + LEN-HARDNESS OF FRAGMENTS

We have seen that the behavior of `WHILEFS` can be expressed in the theory of TWE. A decision procedure for TWE-satisfiability would allow us to perform automatic analysis of `WHILEFS` programs. However, the theory is hard to decide: it encodes the theory of word equations with length constraints (WE + LEN), the decidability of which remains a longstanding open question [11, 21]. In fact, there are several distinct ways it can encode WE + LEN; this section describes these reductions via different fragments of TWE. Figure 7 provides an overview summarizing the reductions between TWE, WE + LEN, WE, and fragments of TWE.

We first demonstrate two fragment reductions that use different features of TWE (§5.1). We start with a reduction that only uses the word equations and normalization (norm) features of TWE—i.e., we reduce to a fragment of the full theory, WE + NORM, that is already sufficient to encode WE + LEN. The reduction to TWE, being a superset of WE + NORM, therefore follows trivially. We then observe that the same structure of reduction is possible using only word equations and walk assertions ($P \rightarrow_V L$), thereby establishing that a second fragment (WE + WALK) can also encode WE + LEN. Together, these results map out which features of TWE contribute to its hardness.

The constructions exploit *unbounded but controlled upward traversal* via ‘..’ and a mechanism for *observation of the root directory* to force equality between the *depths* of walks induced by paths, which corresponds to length equality for certain kinds of walks.

Drawing on these observations, we describe a fragment that likely sits at the boundary of decidability for TWE (§5.2): word equations plus shape constraints. This fragment, WE + SHAPE, omits exactly the features of TWE that WE + NORM and WE + WALK demonstrate contribute (independently) to the problem’s hardness, making it the next clear candidate fragment. The prior reduction strategy does not work for this fragment. However, adding regular constraints (a standard extension axis for word equation theories) recovers the length equalities, so the resulting theory WE + SHAPE + REG is as hard as WE + LEN. These observations suggest that WE + SHAPE may be the most expressive fragment of TWE whose decidability remains in reach.

Preliminaries. Recall that TWE extends WE, the standard theory of ordinary word equations, i.e., $\varphi ::= U = V$ over words U, V only. WE + LEN extends WE with arithmetic predicates over *variable* lengths of the form $c_0 + \sum_i c_i |X_i| = 0$:

$$\begin{aligned} U, V &::= X \mid a \mid \epsilon \mid U \cdot V \\ \Phi &::= U = V \mid (c_0 + \sum_i c_i |X_i| = 0) \end{aligned}$$

In this case, we assume that values are over an infinite alphabet Σ *without* ϵ , $.$, or $..$, i.e., purely ordinary names. The decidability of WE + LEN is open [11, 21]. Note that to reduce WE + LEN to a theory $\mathcal{T}$, it suffices to show a reduction for equality between *word* lengths: $|U| = |V|$ (see Appendix H.1).

5.1 WE + LEN-Hardness of WE + NORM

First, consider the fragment of TWE which extends WE with only syntactic norm. Since there are no trees in this fragment, a model is simply a substitution σ as in ordinary WE. Eliding the standard

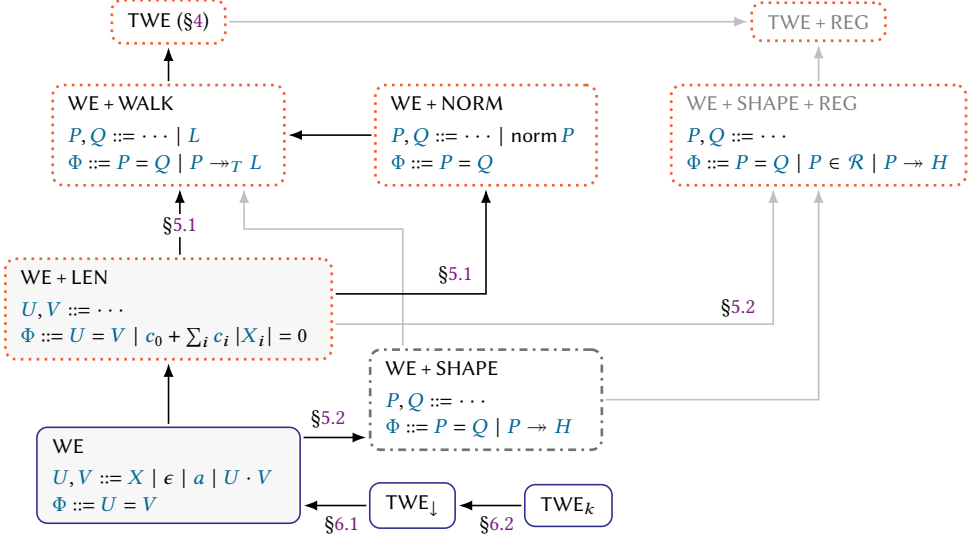


Figure 7. Relationships between **hard** (by reduction of WE + LEN) and **decidable** fragments. $\mathcal{T}_1 \rightarrow \mathcal{T}_2$ means that there is an equisatisfiable reduction of $\mathcal{T}_1$ into $\mathcal{T}_2$, i.e., $\text{SAT}(\mathcal{T}_1) \leq \text{SAT}(\mathcal{T}_2)$. §5 and §6 consider chiefly the fragments on the left and center of the figure; the light-gray ones on the right of the figure are not fragments of TWE, but contextualize the discussed fragments. The decidability of WE + SHAPE (dash-dotted frame) is uncertain.

connectives, WE + NORM has the grammar:

$$\begin{aligned} P, Q &::= X \mid a \mid \epsilon \mid P \cdot Q \mid \text{norm } P \\ \Phi &::= P = Q \end{aligned}$$

There is an equisatisfiable reduction $(\cdot)$ from WE + LEN to this fragment.

THEOREM 2 (WE + LEN reduces to WE + NORM). *There exists $(\cdot)$ such that for any WE + LEN formula Φ , $\text{SAT}_{\text{WE+LEN}}(\Phi)$ if and only if $\text{SAT}_{\text{WE+NORM}}((\Phi))$.*

It suffices to define and prove equisatisfiability for $(\|U_1\| = \|U_2\|)$. The reduction enforces length equality by forcing U_1 and U_2 to represent *anchored walks from the root to the same depth*. Concretely, we reduce U_1 and U_2 to paths P_1 and P_2 which induce net-downward walks that would return to the root via the same net-upward Q , i.e., $P_1 \cdot Q$ and $P_2 \cdot Q$ each induce *perfectly balanced excursions*. Since P_1 and P_2 are balanced by the same Q , they must induce walks to the same depth. The paths (in WE + NORM) must be net-downward but may not be strictly linear; however, for so-called *anchored* paths, equal *depth* means that the *normalized* forms are of equal *length*. By taking the norm of the satisfying values for P_1 and P_2 , we can recover the linear values for U_1, U_2 to reconstruct a satisfying model for WE + LEN.

First, we assume that such *Balanced P* and *Anchored P* predicates are expressible in WE + NORM and demonstrate the reduction. Then we define both and show the desired properties.

5.1.1 Reduction. The idea is to reduce length equalities to coordinated, *perfectly balanced excursions* which force equal *depth*, and recover equal *length* by requiring *anchored* paths. In particular, the

excursions use the same “return” path Q :

$$(|U_1| = |U_2|) \triangleq \text{Balanced } (P_1 \cdot Q) \wedge \text{Balanced } (P_2 \cdot Q) \quad \text{fresh } Q$$

The reduction also conjoins Anchored X for every source variable X . Perfect balance means an anchored excursion of net-zero depth:

PROPERTY 1 (*Perfect balance*).

$$\sigma \models \text{Balanced } P \quad \text{iff} \quad \sigma \models \text{Anchored } P \quad \text{and} \quad \text{depth } \llbracket P \rrbracket_\sigma = 0$$

where $\text{depth } p \in \text{Int}$ counts the (possibly negative) net downward movement induced by p :

$$\begin{aligned} \text{depth } \varepsilon &\triangleq 0 \\ \text{depth } (p/a) &\triangleq \text{depth } p + 1 \\ \text{depth } (p/\epsilon) &\triangleq \text{depth } p \\ \text{depth } (p/.) &\triangleq \text{depth } p \\ \text{depth } (p/..) &\triangleq \text{depth } p - 1 \end{aligned}$$

We will characterize Anchored P precisely below, but, intuitively, it demands that P induce a proper downward walk. By perfect balance, $\sigma \models \text{Balanced } (P_1 \cdot Q) \wedge \text{Balanced } (P_2 \cdot Q)$ implies that both excursions are anchored and have depth 0. Hence, each P_i , as a prefix of its excursion, is anchored, and

$$\begin{aligned} \text{depth } \llbracket P_1 \rrbracket_\sigma + \text{depth } \llbracket Q \rrbracket_\sigma &= \text{depth } \llbracket P_2 \rrbracket_\sigma + \text{depth } \llbracket Q \rrbracket_\sigma, \\ \text{depth } \llbracket P_1 \rrbracket_\sigma &= \text{depth } \llbracket P_2 \rrbracket_\sigma. \end{aligned}$$

The forward direction of theorem 2 is trivial, since values in $\text{WE} + \text{LEN}$ are in Σ^* , i.e., they are effectively linear paths. If $\sigma \models (|U_1| = |U_2|)$, then $\text{depth } \llbracket U_1 \rrbracket_\sigma = \text{depth } \llbracket U_2 \rrbracket_\sigma$, since $|p| = \text{depth } p$ for linear p . The backwards direction is harder: for *non-linear* paths of equal depth in the solution for $\text{WE} + \text{NORM}$, we must recover linear paths of equal length.

Of course, for non-linear paths, equality of depth does not generally imply equality of length. For instance, $a/..a'$ and a have the same depth of 1 but different lengths. Observe, however, that the *normalized* form of a path has length equal to the original depth: $\text{norm } (a/..a') = a'$ and $\text{norm } a = a$ each have length 1. Nonetheless, there sometimes remains a mismatch between depth and normalized length: consider $..a/a'$, which has depth 1 but whose normalized form a/a' has length 2. This mismatch arises because depth always counts $..$ negatively, but norm swallows up $..$ in the same way that $..$ saturates at the root. But the correspondence holds for paths which do not exhibit this saturation—equivalently, whose depth count never goes negative. These are *anchored* paths:

PROPERTY 2 (*Anchoring*).

$$\sigma \models \text{Anchored } P \quad \text{iff} \quad \text{depth } p' \geq 0 \text{ for every prefix } p' \preceq \llbracket P \rrbracket_\sigma$$

For anchored paths, $..$ *always* cancels, hence normalization agrees with depth-counting.

LEMMA 1. *If $\sigma \models \text{Anchored } P$, then $\text{depth } \llbracket P \rrbracket_\sigma = |\text{norm } \llbracket P \rrbracket_\sigma|$.*

As a result, $\sigma \models \text{Balanced } (P_1 \cdot Q) \wedge \text{Balanced } (P_2 \cdot Q)$ gives $|\text{norm } \llbracket P_1 \rrbracket_\sigma| = |\text{norm } \llbracket P_2 \rrbracket_\sigma|$. Hence, to reconstruct equal-length, linear values for U_1 and U_2 , we may simply take $\text{norm } \llbracket P_1 \rrbracket_\sigma$ and $\text{norm } \llbracket P_2 \rrbracket_\sigma$. This completes the backwards direction. It remains only to define *Balanced* and *Anchored* in $\text{WE} + \text{NORM}$.

5.1.2 Defining Balanced and Anchored. The two necessary predicates can be expressed using norm:

$$\begin{aligned} \text{Balanced } P &\triangleq \text{norm } P = \epsilon \wedge \text{norm } (a \cdot P) \neq \epsilon \quad \text{fresh } a \\ \text{Anchored } P &\triangleq \text{norm } P \neq \text{norm } (a \cdot P) \quad \text{fresh } a \end{aligned}$$

The first conjunct for Balanced means that P is an excursion which returns to the root, detected via equality with the empty path. Together with the first conjunct, the second is precisely the disequality used by Anchored P , so it ensures that the excursion never moves above the root.

Anchored P ensures prefixes never have depth less than 0 by preventing saturation in P . If P never attempts to rise above the root, then a is untouched by normalization, so $\text{norm } (a \cdot P) = a \cdot \text{norm } P$ differs from $\text{norm } P$. On the contrary, if it does, then $P = P_1 \cdot P_2$ where P_1 has depth of -1 . In this case, the residual $..$ cancels a , and from that point both sides normalize identically, violating the disequality.

LEMMA 2. *Balanced and Anchored satisfy properties 1 and 2, respectively.*

A similar construction works for the fragment WE + WALK which extends bare WE with walk assertions over abstract trees, using locations in lieu of norm for Balanced and Anchored (see Appendix H.3):

THEOREM 3 (WE + LEN reduces to WE + WALK). *There exists $(\cdot)_\parallel$ such that for any WE + LEN formula Φ , $\text{SAT}_{\text{WE} + \text{LEN}}(\Phi)$ if and only if $\text{SAT}_{\text{WE} + \text{WALK}}((\Phi)_\parallel)$.*

5.2 WE + LEN-Hardness of WE + SHAPE + REG

The constructions for WE + NORM and WE + WALK observe the depth of a path via root detection and transport depth equalities to length equalities via a restriction to anchored paths. Both ingredients rely on reasoning about steps within the walk taken by a path using normalized forms. As such, the construction fails when the ability to examine normalized forms is removed, as in the fragment WE + SHAPE, which features walks to *opaque* locations, $P \rightarrow_V H \triangleq P \rightarrow_V L \in H$, where L is always fresh and inaccessible:

$$\begin{aligned} P, Q &::= X \mid a \mid \epsilon \mid P \cdot Q \\ \Phi &::= P = Q \mid P \rightarrow_V H \end{aligned}$$

Without access to norm or L , it is no longer clear how to detect that a walk goes to the root directory or to restrict solutions to anchored paths. Even if precise root isolation could be achieved using shape assertions, it is unclear how non-anchored paths could be eliminated.

Interestingly, length equalities can be recovered if regular constraints on paths are added to this fragment. The regular constraints in WE + SHAPE + REG can be represented by symbolic automata over the infinite alphabet of path components. The regular constraints put paths into a rigid form in which a block of $..$ components can serve as a counter for the length of a word. Shape assertions then rule out the wrong counter lengths: if the block is too short or too long, the constraints force incompatible file and directory requirements. We leave the precise proof to Appendix H.4.

6 DECIDABILITY OF LINEAR AND BOUNDED THEORY FRAGMENTS

While the full theory of TWE is hard to decide, the reliance of the reductions in the previous section on unbounded upwards movement points to useful decidable fragments. In §6.1 we show decidability of the *linear* fragment $\text{TWE}_\downarrow$, which disallows non-linear components in paths and thus only permits downward movement. Then, §6.2 exploits this engine to decide a *bounded* non-linear fragment TWE_k in which the number of occurrences of non-linear components in each path is bounded by a constant k .

Unlike the fragments in the previous section, these are *semantic restrictions* on models, not on the syntax of TWE-formulae or WHILE_{FS} programs. They are nonetheless well-motivated and practically realizable: in practice paths are often linear or exhibit a limited number of $\cdot$ components. File system interaction bugs may be discovered using reasonably low bounds; for instance, the buggy execution of `rmr` in listing 2 requires only $k = 1$. Moreover, the reduction to word equations allows leveraging efficient solvers which support sequence theories, such as Z3 [17] or cvc5 [6].

6.1 Decidability of linear TWE

First, we consider $\text{TWE}_{\downarrow}$, a fragment of TWE which allows only models which do not exhibit non-linear components in any path. Precisely, a model $\langle \sigma, \theta \rangle$ is within the fragment if, for all P which appear in the formula Φ , the evaluation of P is linear, i.e., $\llbracket P \rrbracket_{\sigma, \theta}$ lin. Note that all constant components in Φ must be linear as a result. For instance, for the formula $V[A \cdot X] \in \text{dir}$, $\text{TWE}_{\downarrow}$ admits a model mapping X to ε or a' but not one mapping it to ϵ , $\cdot$, or ...

Since every path is linear, every walk $P \rightarrow_T L$ follows ordinary child components through the tree; indeed, P is identical to L . In this setting, tree reasoning can be compiled into pure word constraints.

The target WE theory is over the infinite alphabet of names and distinguishes between *word variables* standing for words and *letter variables* for single letters. Its atoms are word and name equations. Assuming that equality of names is decidable in PSPACE, the target fragment is decidable in PSPACE [31, Theorem 3].

THEOREM 4. *There exists an equisatisfiable PSPACE reduction from $\text{TWE}_{\downarrow}$ to WE.*

COROLLARY 1. *Satisfiability of $\text{TWE}_{\downarrow}$ is decidable in PSPACE.*

The proof, illustrated in figure 8, proceeds via a series of equisatisfiable reductions which progressively rewrite features from the theory, until pure word constraints are reached. The key fact is that linear paths collapse to locations; we rewrite walks into shape assertions over tree selections (§6.1.1). The remaining features—tree updates, entries sets, and shape consistency—reason *locally* with respect to *ancestors*: each atom depends only on the ancestor and descendant relationships between locations, which are word-expressible using prefix predicates.

Concretely, we eliminate tree updates $T[L \mapsto T']$, so that all trees are selections over abstract trees $V[L]$ only (§6.1.2). Then, we eliminate entries T into words whose elements must be mutually

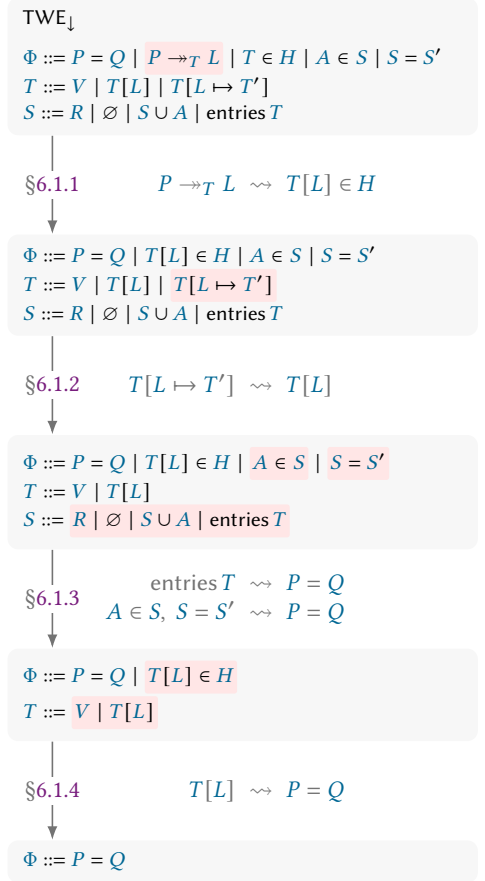


Figure 8. **Reduction of $\text{TWE}_{\downarrow}$ to WE.** Each step eliminates one or more features of $\text{TWE}_{\downarrow}$ (highlighted red) by rewriting them, eventually ending up with only word equations.

distinct (§6.1.3). This property is not typically expressible over words of unbounded length, but since linear walks traverse only one location at each depth-level in the tree, it is possible to establish a small model property for the words and express mutual distinction via a bounded number of pair-wise conjunctions. Finally, trees are eliminated entirely: shape assertions over selections are rewritten via a topological analysis into prefix constraints (§6.1.4). Here, we present the construction; Appendix I details a full proof.

6.1.1 Rewriting walk assertions. Since paths coincide with locations in the linear setting, walks coincide with tree selections. It is therefore possible to dispense with walk assertions altogether by rewriting them into shape assertions over tree selections. Specifically,

$$P \twoheadrightarrow_T L \Leftrightarrow (P = L) \wedge (T[L] \in \{\text{dirc}, \text{file}, \perp\}) \quad \text{where } T \in \mathcal{H} \triangleq \bigvee_{H \in \mathcal{H}} T \in H$$

The crucial fact here is linearity; in the general case the path can traverse locations unrelated to L , making such a rewrite unsound.

6.1.2 Eliminating tree updates. We eliminate $T[L_1 \mapsto T']$ inductively, such that only selections over abstract trees remain. In essence, we can capture the effect of updates on selections based on if the selection falls inside or outside the updated sub-tree. For a selection at an arbitrary location L , we split into two cases: (1) if the selection falls within the updated subtree, it proceeds into the subtree; (2) otherwise, it observes the original tree. Writing $L \preceq L'$ to denote that L is a prefix of L' , for shape constraints:

$$T[L_1 \mapsto T'] [L] \in H \iff ((L = L_1 \cdot L_2) \wedge (T'[L_2] \in H)) \vee ((L_1 \not\preceq L) \wedge (T[L] \in H))$$

Likewise, we must rewrite entries over updates. Again, since the effect of the update is local, and the set of entries concerns only the *immediate children* of a tree, an update at L_1 may affect it in three ways depending on the ancestor relationship between the queried and updated locations. (1) If the directory queried at L is inside the updated sub-tree T' (i.e., $L = L_1 \cdot L_2$), then the updated directory $T'[L_2]$ is used. Otherwise, (2) if it is outside the updated sub-tree, the entry set is unaffected—*except* (3) if the update is at a direct child (i.e., $L \cdot A = L_1$), in which case the update adds or removes that child’s name A . Precisely, we replace entries $T[L_1 \mapsto T'] [L]$ with an abstract set R with a condition given by $\Psi_{\text{inside}} \vee \Psi_{\text{outside}} \vee \Psi_{\text{child}}$:

$$\begin{aligned} \Psi_{\text{inside}} &\triangleq L_1 \cdot L_2 = L \wedge R = \text{entries } T'[L_2] \\ \Psi_{\text{outside}} &\triangleq \left(\begin{array}{l} L_1 = L \cdot A \cdot A' \cdot L_2 \\ \vee L \parallel L_1 \end{array} \right) \wedge R = \text{entries } T[L] \\ \Psi_{\text{child}} &\triangleq L_1 = L \cdot A \wedge \left(\begin{array}{l} (T' \in \{\text{file}, \text{dirc}\}) \wedge R = (\text{entries } T[L]) \cup A \\ \vee (T' \in \perp \wedge A \notin R \wedge R \cup A = (\text{entries } T[L]) \cup A) \end{array} \right) \end{aligned}$$

We write $L \parallel L'$ to mean that neither is a prefix of the other.

At this point, all tree terms are nested selections over variables $V[L_1] \cdots [L_k]$. We may safely flatten each of these chains into a single selection $V[L_1 \cdots L_k]$.

6.1.3 Reducing entries into bounded-length words. After elimination of tree updates, every entries term has the form $\text{entries } V[L]$. We eliminate such terms first, observing that the set of names admits finite description based on the selections witnessed in the formula. Then, we encode name sets S into words whose elements must be mutually distinct. This property is not generally expressible in word equations for words of *unbounded* length, but it is possible to establish a small model property on name sets and enforce mutual distinction using polynomially many disequalities.

Eliminating entries. Observe that $\llbracket \text{entries } V[L] \rrbracket_{\sigma, \theta}$ is effectively the set of immediate child names $\{a \mid \langle \sigma, \theta \rangle \models V[L \cdot a] \in \{\text{file}, \text{dir}\}\}$. This set, finite yet unbounded, admits a small model property. Only those *names witnessed by some selection on V* are pinned; all others are free. Hence it suffices to replace entries terms with abstract name sets R equipped with guarded membership predicates ranging over the finite set of selection locations $L \in \mathcal{L}_V(\Phi) = \{L \mid V[L] \text{ in } \Phi\}$. In particular, we assert $\Psi_{\{\text{file}, \perp\}} \vee \Psi_{\text{dir}}$:

$$\begin{aligned} \Psi_{\{\text{file}, \perp\}} &\triangleq V[L] \in \{\text{file}, \perp\} \wedge R = \emptyset \\ \Psi_{\text{dir}} &\triangleq V[L] \in \text{dir} \wedge \left(\bigwedge_{L' \in \mathcal{L}_V(\Phi)} \Psi_{\text{outside}} \vee \Psi_{\text{child}} \vee \Psi_{\text{descendant}} \right). \end{aligned}$$

In essence, files and absent locations have no entries, but for directories, we again perform prefix-based case analysis to determine if a selection at L' contributes to pinned names at $V[L]$:

$$\begin{aligned} \Psi_{\text{outside}} &\triangleq L' = L \vee (L \not\prec L') \\ \Psi_{\text{child}} &\triangleq L' = L \cdot A \wedge \left(\begin{array}{l} (V[L'] \in \{\text{file}, \text{dir}\} \wedge A \in R) \\ \vee (V[L'] \in \perp \wedge A \notin R) \end{array} \right) \\ \Psi_{\text{descendant}} &\triangleq L' = L \cdot A \cdot A' \cdot L'' \wedge A \in R \end{aligned}$$

If the selection is outside of the strict descendants of $V[L]$, then it is outside the region of interest. The selection only contributes to R if it is a file or directory at the child A of $V[L]$, in which case $A \in R$; or if it is a strict descendant, in which case some entry A must exist. Additionally, to preserve functional consistency, for any two pairs of entries $\text{entries } V[L_1]$ and $\text{entries } V[L_2]$, we assert that $L_1 = L_2 \Rightarrow R_1 = R_2$.

Reducing name sets into words. Now, for name sets, only assertions for equality $S_1 = S_2$, membership $A \in S$, and non-membership $A \notin S$ over $S ::= R \mid \emptyset \mid S \cup A$ remain. A standard small model property enables a reduction of sets to words. Because only membership predicates and insertions pin set members, the sets for satisfying models can be pruned to those with cardinality of at most $|\mathcal{S}(\Phi)|$, where $\mathcal{S}(\Phi)$ is the set $\{A \mid A \in S \text{ or } S \cup A \text{ in } \Phi\}$. Then, each set variable R is replaced by a sequence of fresh name variables $X_{A_1} \cdots X_{A_{|\mathcal{S}(\Phi)|}}$ which are pairwise distinct:

$$\bigwedge_{1 \leq i < j \leq |\mathcal{S}(\Phi)|} A_i \neq A_j.$$

The other constructors and predicates are rewritten into words in the obvious way. Note that we use a fresh sentinel name to allow the sequence to represent a set of cardinality less than $|\mathcal{S}(\Phi)|$ (see Appendix I.3 for full details).

6.1.4 Reifying shape lookups into prefix analysis. Once left with word equations and shape assertions over tree selections $V[L] \in H$, the question is one of realizability: is there an arrangement of the locations such that the shapes describe a valid tree? The sole structural invariant is that all files and absent locations must be in leaf positions, and this property is readily expressed in terms of word-expressible ancestorship predicates: if $V[L] \in \{\text{file}, \perp\}$, then for any other selection $V[L'] \in H$, L' must not be a strict descendant of L : $L \not\prec L'$. As with entries, functional consistency asserts that $L_1 = L_2 \Rightarrow H_1 = H_2$.

The resulting formula is in WE, which is decidable in PSPACE.

6.2 Decidability of bounded non-linear TWE

A decision procedure for the linear fragment $\text{TWE}_\downarrow$ enables decidability of a *bounded* non-linear fragment TWE_k . This restriction admits models where each path may exhibit up to k -many non-linear components, where $k \geq 0$ is a constant. For each path, the constant bound permits a decomposition into $m + 1$ linear paths delimited by m non-linear components, for some $m \leq k$. Then, each non-linear step of a walk can be handled explicitly via case work and simulated via linear walks and shape assertions in $\text{TWE}_\downarrow$.

THEOREM 5. *Given a TWE_k formula Φ with bound k , there is an equisatisfiable reduction to $\text{TWE}_\downarrow$ and hence to WE using space polynomial in $|\Phi| + k$.*

COROLLARY 2. *Satisfiability of a TWE_k formula Φ with bound k is decidable in space polynomial in $|\Phi| + k$, and hence is in PSPACE for every fixed k .*

For each path P , we know that $P = X_1 \cdot B_1 \cdot X_2 \cdot B_2 \cdots B_m \cdot X_{m+1}$ for some $m \leq k$, where each X_i is linear and each $B_i \in \{\epsilon, \cdot, \cdot\}$. To reduce a walk $P \rightarrow_T L$ to $\text{TWE}_\downarrow$, we introduce fresh linear variables Y_i for $1 \leq i \leq m + 1$ and Z_i for $1 \leq i \leq m$, with the intention that Y_i will correspond to the location reached after resolving the prefix through X_i and Z_i the result of resolving B_i as well. These are defined mutually by induction on i and disjunctive cases on B_i (see Appendix I.6 for the full construction).

Since P resolves successfully to L when Y_i is a directory location for every $1 \leq i \leq m$ and $Y_{m+1} = L$, we may write $P \rightarrow_T L$ using linear walks and shape assertions:

$$Y_{m+1} \rightarrow_T L \wedge \left(\bigwedge_{1 \leq i \leq m} T[Y_i] \in \text{dirc} \right).$$

For norm P , we rewrite with Y_{m+1} .

7 RELATED WORK

Logical foundations. Our framework shares ingredients in common with several areas of prior art—including equations over words, tree navigation and reasoning, and functional updates—and draws them into a single theory suitable for reasoning about file system interactions. We discuss these lines of work below.

Word equations. TWE includes word equations (WE), which enjoy a rich body of literature. Makanin [40] (see also Diekert [19]) first solved the satisfiability problem, and the proposed algorithm was gradually improved to EXPSPACE [25, 32, 50] and eventually to PSPACE [30, 46]. A number of notable extensions to WE are also well-studied, including the addition of regular constraints [50] (REG, decidable in PSPACE) and length constraints [21, 38] (decidability open), as well as restrictions such as quadratic equations [20], straight-line [37] and chain-free fragments [2, 37], and pattern languages [4]. Word equations over infinite alphabets with constraints have also been studied, with decidability and complexity bounds established for several fragments [31]. TWE fits into this line of work as a family of extensions to WE in which words may also induce walks navigating over finite trees. This is in contrast to context unification (decidable in PSPACE [29]), which directly generalizes words into one-holed tree contexts. The cancellation behavior of $\cdot$ in walks bears some resemblance to free inverse monoids [18], though the inverse is one-sided and saturates at root.

Feature logics. The tree elements of TWE share commonalities with several flavors of feature logics. FT [5] is a theory of finite trees with named *features*, which correspond to *names* in our setting. CFT [52] extends FT with arity constraints expressing that a tree has exactly some *constant* set of

features; TWE instead supports reasoning about the entry names of a directory as an unbounded, symbolic set. Like EF [56], an extension of CFT with first-class features, TWE treats *names* as first-class values; however, they are also subject to word equations. The CoLiS project [7] explores file system reasoning using a decidable first-order theory of feature trees extended with functional updates [28] similar to those in TWE. But whereas paths in CoLiS are strictly concrete and linear, our framework supports reasoning about paths with unknown variables, non-linear elements, and equational constraints. Path resolution may be understood as *iterated* feature selection, which is not supported by any existing feature logic.

Tree-walking automata. Tree-walking automata [3], finite state machines which recognize tree languages by walking over an input tree in a manner similar to string automata, have been studied extensively [9]. The procedural perspective on path resolution is directly aligned with tree-walking automata: every concrete path can be understood as a deterministic TWA which accepts file trees in which the path successfully resolves. TWE considers not only concrete paths but also symbolic ones—making some part of the automaton unknown (similar to nested TWAs [55])—but it also adds constraints over those unknown portions via word equations.

XPath logics. XPath [58] is a query language for extracting portions of XML trees, and several lines of work model its navigational core with tree logics and tree-walking automata [8, 55]. In general, XPath queries are more expressive than file system paths, supporting data-value predicates within the path itself (e.g., field *X* of a node satisfies some condition). Although XPath Update [59] provides an update facility to mutate the XML tree, existing logics do not model updates, focusing on selection over a fixed tree. TWE explicitly supports updates.

Separation logics. SL [47] offers a framework for compositional reasoning over resource heaps based on splitting heaps into disjoint fragments. Hierarchical file systems can be understood as memory heaps and paths as addresses (§2.1). Indeed, Gardner et al. [22], drawing on context logic [13], investigate program logics based on structural SL for file system reasoning over *linear* paths. However, non-linear components pose a challenge for disjoint reasoning by introducing non-local walk behavior. To handle non-linear paths, effect fusion logic [45] introduces an effect separating conjunction and effect frame rule in order to propagate updates across heap fragments. Our framework does not aim to develop a program logic; rather, TWE aims to be a basis for first-order automated and semi-automated analyses by isolating the constraint problem generated by path resolution and tree reasoning.

Reasoning about file systems. A complementary body of work focuses on the file system domain, including the correctness of file system implementations and analyses of client programs.

Verified file systems. Verified file systems [14, 15, 51] are file system *implementations* proven correct with respect to a specification, aiming to offer correctness guarantees in the implementation of operations such as file creation or writing. A related line of work formally pins down the semantics of the operations themselves; SibylFS [48] is an executable specification of the POSIX file system interface used to test implementations for conformance. These concerns are complementary to our framework, which targets reasoning about the correctness of *client programs using a file system*, assuming that the implementation is correct.

Static analyses. Several static analysis systems reason about file system interactions to some degree. ShellCheck [26] is a linter that flags *syntactic* patterns corresponding to code smells, some of which involve likely-buggy file system interactions (e.g., deletions of important system paths). In contrast, our framework targets *semantic* reasoning about program execution. CoLiS [7, 28] analyzes Debian package installation scripts but is restricted to concrete and linear paths. SaSh [34, 35] performs

symbolic execution to find bugs in shell programs using a coarse, non-hierarchical model of the file system. FiFA [49] uses abstract interpretation over path values to extract file system state pre-conditions required for a program’s correct execution, but paths are treated as flat strings rather than as modeling resolution over a hierarchical tree. The TWE framework provides formal foundations that can enable the enrichment of these analyses to support hierarchical file systems and symbolic, non-linear paths.

Effect systems. Type-and-effect systems [24, 39, 54], including algebraic effect handlers [36], encode over-approximations of effectful computations into a static typing analysis. The granularity of tracked effects is the kind of effect a program term may produce—e.g., a file write or a file read—not the affected paths. In contrast, effect capability systems [10] with capabilities over file/directory paths [16, 23] treat paths as opaque tokens and do not model path resolution. In principle, TWE can serve as a predicate language for such systems to enable reasoning about path relationships and resolution behavior.

8 FUTURE DIRECTIONS

WHILEFS and TWE provide foundations for further investigation in multiple directions. One is to study the decidability of richer fragments of TWE, including the open fragment WE + SHAPE (§5.2), and TWE with standard restrictions on word equation structure such as straight-line [37] and chain-free fragments [2, 37]. Since the decidable fragments in this paper reduce to WE, automated and semi-automated verifiers and bug-finders based on WHILEFS’s symbolic semantics could use off-the-shelf sequence theory solvers [6, 17] to reason about real file system-interacting code. Finally, the TWE framework itself presents several opportunities for extension to support reasoning about node-level enrichments such as permissions, structural enrichments like soft links and mounts, concurrent and networked file systems, and OS-specific semantics.

REFERENCES

- [1] Michael Gordon Abbott, Thorsten Altenkirch, Conor McBride, and Neil Ghani. 2005. δ for Data: Differentiating Data Structures. *Fundam. Informaticae* 65, 1-2 (2005), 1–28. <http://content.iospress.com/articles/fundamenta-informaticae/f65-1-2-02>
- [2] Parosh Aziz Abdulla, Mohamed Faouzi Atig, Bui Phi Diep, Luk á Holík, and Petr Janku. 2019. Chain-Free String Constraints. In *Automated Technology for Verification and Analysis - 17th International Symposium, ATVA 2019, Taipei, Taiwan, October 28-31, 2019, Proceedings (Lecture Notes in Computer Science, Vol. 11781)*, Yu-Fang Chen, Chih-Hong Cheng, and Javier Esparza (Eds.). Springer, 277–293. https://doi.org/10.1007/978-3-030-31784-3_16
- [3] Alfred V. Aho and Jeffrey D. Ullman. 1971. Translations on a Context-Free Grammar. *Inf. Control* 19, 5 (1971), 439–475. [https://doi.org/10.1016/S0019-9958\(71\)90706-6](https://doi.org/10.1016/S0019-9958(71)90706-6)
- [4] Dana Angluin. 1980. Finding Patterns Common to a Set of Strings. *J. Comput. Syst. Sci.* 21, 1 (1980), 46–62. [https://doi.org/10.1016/0022-0000\(80\)90041-0](https://doi.org/10.1016/0022-0000(80)90041-0)
- [5] Rolf Backofen and Gert Smolka. 1993. A Complete and Recursive Feature Theory. In *31st Annual Meeting of the Association for Computational Linguistics, 22-26 June 1993, Ohio State University, Columbus, Ohio, USA, Proceedings*, Lenhart K. Schubert (Ed.). ACL, 193–200. <https://doi.org/10.3115/981574.981600>
- [6] Haniel Barbosa, Clark Barrett, Martin Brain, Gereon Kremer, Hanna Lachnitt, Makai Mann, Abdalrhman Mohamed, Mudathir Mohamed, Aina Niemetz, Andres Nötzli, Alex Ozdemir, Mathias Preiner, Andrew Reynolds, Ying Sheng, Cesare Tinelli, and Yoni Zohar. 2022. cvc5: A Versatile and Industrial-Strength SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems*, Dana Fisman and Grigore Rosu (Eds.). Springer International Publishing, Cham, 415–442.
- [7] Benedikt F. H. Becker, Nicolas Jeannerod, Claude Marché, Yann Régis-Gianas, Mihaela Sighireanu, and Ralf Treinen. 2022. The CoLiS platform for the analysis of maintainer scripts in Debian software packages. *Int. J. Softw. Tools Technol. Transf.* 24, 5 (2022), 717–733. <https://doi.org/10.1007/S10009-022-00671-1>
- [8] Michael Benedikt and Christoph Koch. 2008. XPath leashed. *ACM Comput. Surv.* 41, 1 (2008), 3:1–3:54. <https://doi.org/10.1145/1456650.1456653>

- [9] Mikolaj Bojanczyk. 2008. Tree-Walking Automata. In *Language and Automata Theory and Applications, Second International Conference, LATA 2008, Tarragona, Spain, March 13-19, 2008. Revised Papers (Lecture Notes in Computer Science, Vol. 5196)*, Carlos Martín-Vide, Friedrich Otto, and Henning Fernau (Eds.). Springer, 1–2. https://doi.org/10.1007/978-3-540-88282-4_1
- [10] Jonathan Immanuel Brachthäuser, Philipp Schuster, and Klaus Ostermann. 2020. Effects as capabilities: effect handlers and lightweight effect polymorphism. *Proc. ACM Program. Lang.* 4, OOPSLA (2020), 126:1–126:30. <https://doi.org/10.1145/3428194>
- [11] J. Richard Büchi and Steven Senger. 1988. Definability in the Existential Theory of Concatenation and Undecidable Extensions of this Theory. *Math. Log. Q.* 34, 4 (1988), 337–342. <https://doi.org/10.1002/MALQ.19880340410>
- [12] BusyBox. [n. d.]. BusyBox: The Swiss Army Knife of Embedded Linux. <https://www.busybox.net/>.
- [13] Cristiano Calcagno, Philippa Gardner, and Uri Zarfaty. 2005. Context logic and tree update. In *Proceedings of the 32nd ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2005, Long Beach, California, USA, January 12-14, 2005*, Jens Palsberg and Martin Abadi (Eds.). ACM, 271–282. <https://doi.org/10.1145/1040305.1040328>
- [14] Haogang Chen, Tej Chajed, Alex Konradi, Stephanie Wang, Atalay Mert Ileri, Adam Chlipala, M. Frans Kaashoek, and Nikolai Zeldovich. 2017. Verifying a high-performance crash-safe file system using a tree specification. In *Proceedings of the 26th Symposium on Operating Systems Principles, Shanghai, China, October 28-31, 2017*. ACM, 270–286. <https://doi.org/10.1145/3132747.3132776>
- [15] Haogang Chen, Daniel Ziegler, Tej Chajed, Adam Chlipala, M. Frans Kaashoek, and Nikolai Zeldovich. 2015. Using Crash Hoare logic for certifying the FSCQ file system. In *Proceedings of the 25th Symposium on Operating Systems Principles, SOSP 2015, Monterey, CA, USA, October 4-7, 2015*, Ethan L. Miller and Steven Hand (Eds.). ACM, 18–37. <https://doi.org/10.1145/2815400.2815402>
- [16] Aaron Craig, Alex Potanin, Lindsay Groves, and Jonathan Aldrich. 2018. Capabilities: Effects for Free. In *Formal Methods and Software Engineering - 20th International Conference on Formal Engineering Methods, ICFEM 2018, Gold Coast, QLD, Australia, November 12-16, 2018, Proceedings (Lecture Notes in Computer Science, Vol. 11232)*, Jing Sun and Meng Sun (Eds.). Springer, 231–247. https://doi.org/10.1007/978-3-030-02450-5_14
- [17] Leonardo de Moura and Nikolaj Bjørner. 2008. Z3: An Efficient SMT Solver. In *Tools and Algorithms for the Construction and Analysis of Systems, C. R. Ramakrishnan and Jakob Rehof (Eds.)*. Springer Berlin Heidelberg, Berlin, Heidelberg, 337–340.
- [18] Timothy Deis, John Meakin, and G. Sénizergues. 2007. Equations in Free Inverse Monoids. *International Journal of Algebra and Computation* 17, 04 (June 2007), 761–795. <https://doi.org/10.1142/S0218196707003755>
- [19] Volker Diekert. 2015. More Than 1700 Years of Word Equations. In *Algebraic Informatics - 6th International Conference, CAI 2015, Stuttgart, Germany, September 1-4, 2015. Proceedings (Lecture Notes in Computer Science, Vol. 9270)*, Andreas Maletti (Ed.). Springer, 22–28. https://doi.org/10.1007/978-3-319-23021-4_2
- [20] Volker Diekert and John Michael Robson. 1999. Quadratic Word Equations. In *Jewels are Forever, Contributions on Theoretical Computer Science in Honor of Arto Salomaa*, Juhani Karhumäki, Hermann A. Maurer, Gheorghe Paun, and Grzegorz Rozenberg (Eds.). Springer, 314–326.
- [21] Vijay Ganesh, Mia Minnes, Armando Solar-Lezama, and Martin C. Rinard. 2012. Word Equations with Length Constraints: What’s Decidable?. In *Hardware and Software: Verification and Testing - 8th International Haifa Verification Conference, HVC 2012, Haifa, Israel, November 6-8, 2012. Revised Selected Papers (Lecture Notes in Computer Science, Vol. 7857)*, Armin Biere, Amir Nahir, and Tanja E. J. Vos (Eds.). Springer, 209–226. https://doi.org/10.1007/978-3-642-39611-3_21
- [22] Philippa Gardner, Gian Ntzik, and Adam Wright. 2014. Local Reasoning for the POSIX File System. In *Programming Languages and Systems - 23rd European Symposium on Programming, ESOP 2014, Held as Part of the European Joint Conferences on Theory and Practice of Software, ETAPS 2014, Grenoble, France, April 5-13, 2014, Proceedings (Lecture Notes in Computer Science, Vol. 8410)*, Zhong Shao (Ed.). Springer, 169–188. https://doi.org/10.1007/978-3-642-54833-8_10
- [23] Isaac Oscar Gariano, James Noble, and Marco Servetto. 2019. CallE: An Effect System for Method Calls. *CoRR abs/1907.04934* (2019). <http://arxiv.org/abs/1907.04934>
- [24] David K. Gifford and John M. Lucassen. 1986. Integrating Functional and Imperative Programming. In *Proceedings of the 1986 ACM Conference on LISP and Functional Programming, LFP 1986, Cambridge, Massachusetts, USA, August 4-6, 1986*, William L. Scherlis, John H. Williams, and Richard P. Gabriel (Eds.). ACM, 28–38. <https://doi.org/10.1145/319838.319848>
- [25] Claudio Gutierrez. 1998. Satisfiability of Word Equations with Constants is in Exponential Space. In *39th Annual Symposium on Foundations of Computer Science, FOCS 1998, Palo Alto, California, USA, November 8-11, 1998*. IEEE Computer Society, 112–119. <https://doi.org/10.1109/SFCS.1998.743434>
- [26] Vidar Holen. 2026. ShellCheck. <https://www.shellcheck.net/>.
- [27] Gérard P. Huet. 1997. The Zipper. *J. Funct. Program.* 7, 5 (1997), 549–554. <https://doi.org/10.1017/S0956796897002864>
- [28] Nicolas Jeannerod and Ralf Treinen. 2018. Deciding the First-Order Theory of an Algebra of Feature Trees with Updates. In *Automated Reasoning - 9th International Joint Conference, IJCAR 2018, Held as Part of the Federated Logic Conference,*

- FloC 2018, Oxford, UK, July 14-17, 2018, Proceedings (Lecture Notes in Computer Science, Vol. 10900)*, Didier Galmiche, Stephan Schulz, and Roberto Sebastiani (Eds.). Springer, 439–454. https://doi.org/10.1007/978-3-319-94205-6_29
- [29] Artur Jež. 2014. Context Unification is in PSPACE. In *Automata, Languages, and Programming - 41st International Colloquium, ICALP 2014, Copenhagen, Denmark, July 8-11, 2014, Proceedings, Part II (Lecture Notes in Computer Science, Vol. 8573)*, Javier Esparza, Pierre Fraigniaud, Thore Husfeldt, and Elias Koutsoupias (Eds.). Springer, 244–255. https://doi.org/10.1007/978-3-662-43951-7_21
- [30] Artur Jež. 2016. Recompression: A Simple and Powerful Technique for Word Equations. *J. ACM* 63, 1 (2016), 4:1–4:51. <https://doi.org/10.1145/2743014>
- [31] Artur Jež, Anthony W Lin, Oliver Markgraf, and Philipp Rümmer. 2023. Decision Procedures for Sequence Theories. In *International Conference on Computer Aided Verification*. Springer, 18–40. https://doi.org/10.1007/978-3-031-37703-7_2
- [32] Antoni Koscielski and Leszek Pacholski. 1996. Complexity of Makanin’s Algorithm. *J. ACM* 43, 4 (1996), 670–684. <https://doi.org/10.1145/234533.234543>
- [33] Frank Landymore. 2026. Blundering Husband Asks Claude AI to “Organize” Wife’s PC, Accidentally Erases Her Cherished Family Photos. <https://futurism.com/artificial-intelligence/claude-wife-photos>.
- [34] Lukas Lazarek, Seong-Heon Jung, Evangelos Lamprou, Zekai Li, Anirudh Narsipur, Eric Zhao, Michael Greenberg, Konstantinos Kallas, Konstantinos Mamouras, and Nikos Vasilakis. 2025. From Ahead-of- to Just-in-Time and Back Again: Static Analysis for Unix Shell Programs. In *Proceedings of the 2025 Workshop on Hot Topics in Operating Systems, HotOS 2025, Banff, AB, Canada, May 14-16, 2025*. ACM, 88–95. <https://doi.org/10.1145/3713082.3730395>
- [35] Lukas Lazarek, Evangelos Lamprou, George Kapetenakis, Eric Zhao, Zhiwen Zheng, Michael Greenberg, Konstantinos Kallas, and Nikos Vasilakis. 2026. Ahead-of-time Analysis of Shell Program Effects. In *Proceedings of the ACM SIGOPS 32nd Symposium on Operating Systems Principles, SOSP 2026, Clarion Congress Hotel Prague, Prague, Czechia, September 29-October 2, 2026 (SOSP ’26)*. Association for Computing Machinery, New York, NY, USA.
- [36] Daan Leijen. 2014. Koka: Programming with Row Polymorphic Effect Types. In *Proceedings 5th Workshop on Mathematically Structured Functional Programming, MSFP@ETAPS 2014, Grenoble, France, 12 April 2014 (EPTCS, Vol. 153)*, Paul Blain Levy and Neel Krishnaswami (Eds.). 100–126. <https://doi.org/10.4204/EPTCS.153.8>
- [37] Anthony Widjaja Lin and Pablo Barceló. 2016. String solving with word equations and transducers: towards a logic for analysing mutation XSS. In *Proceedings of the 43rd Annual ACM SIGPLAN-SIGACT Symposium on Principles of Programming Languages, POPL 2016, St. Petersburg, FL, USA, January 20 - 22, 2016*, Rastislav Bodík and Rupak Majumdar (Eds.). ACM, 123–136. <https://doi.org/10.1145/2837614.2837641>
- [38] Anthony W. Lin and Rupak Majumdar. 2018. Quadratic Word Equations with Length Constraints, Counter Systems, and Presburger Arithmetic with Divisibility. In *Automated Technology for Verification and Analysis - 16th International Symposium, ATVA 2018, Los Angeles, CA, USA, October 7-10, 2018, Proceedings (Lecture Notes in Computer Science, Vol. 11138)*, Shuvendu K. Lahiri and Chao Wang (Eds.). Springer, 352–369. https://doi.org/10.1007/978-3-030-01090-4_21
- [39] John M. Lucassen and David K. Gifford. 1988. Polymorphic Effect Systems. In *Conference Record of the Fifteenth Annual ACM Symposium on Principles of Programming Languages, San Diego, California, USA, January 10-13, 1988*, Jeanne Ferrante and Peter Mager (Eds.). ACM Press, 47–57. <https://doi.org/10.1145/73560.73564>
- [40] G. S. Makanin. 1977. The problem of solvability of equations in a free semigroup. *Mathematics of the USSR-Sbornik* 32, 2 (1977), 129–198.
- [41] Conor McBride. 2001. The Derivative of a Regular Type is its Type of One-Hole Contexts. (2001). <http://strictlypositive.org/diff.pdf>
- [42] John McCarthy. 1962. Towards a Mathematical Science of Computation. In *Information Processing, Proceedings of the 2nd IFIP Congress 1962, Munich, Germany, August 27 - September 1, 1962*. North-Holland, 21–28.
- [43] MITRE Corporation. 2024. CWE-22: Improper Limitation of a Pathname to a Restricted Directory (‘Path Traversal’). Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/22.html>
- [44] MITRE Corporation. 2024. CWE-23: Relative Path Traversal. Common Weakness Enumeration. <https://cwe.mitre.org/data/definitions/23.html>
- [45] Gian Ntzik and Philippa Gardner. 2015. Reasoning about the POSIX file system: local update and global pathnames. In *Proceedings of the 2015 ACM SIGPLAN International Conference on Object-Oriented Programming, Systems, Languages, and Applications, OOPSLA 2015, part of SPLASH 2015, Pittsburgh, PA, USA, October 25-30, 2015*, Jonathan Aldrich and Patrick Eugster (Eds.). ACM, 201–220. <https://doi.org/10.1145/2814270.2814306>
- [46] Wojciech Plandowski. 2004. Satisfiability of word equations with constants is in PSPACE. *J. ACM* 51, 3 (2004), 483–496. <https://doi.org/10.1145/990308.990312>
- [47] John C. Reynolds. 2002. Separation Logic: A Logic for Shared Mutable Data Structures. In *17th IEEE Symposium on Logic in Computer Science (LICS 2002), 22-25 July 2002, Copenhagen, Denmark, Proceedings*. IEEE Computer Society, 55–74. <https://doi.org/10.1109/LICS.2002.1029817>
- [48] Tom Ridge, David Sheets, Thomas Tuerk, Andrea Giugliano, Anil Madhavapeddy, and Peter Sewell. 2015. SibylFS: formal specification and oracle-based testing for POSIX and real-world file systems. In *Proceedings of the 25th Symposium on*

- Operating Systems Principles, SOSP 2015, Monterey, CA, USA, October 4-7, 2015*, Ethan L. Miller and Steven Hand (Eds.). ACM, 38–53. <https://doi.org/10.1145/2815400.2815411>
- [49] Rodney Rodriguez and Xiaoyin Wang. 2021. Understanding Execution Environment of File-Manipulation Scripts by Extracting Pre-Conditions. In *29th IEEE/ACM International Conference on Program Comprehension, ICPC 2021, Madrid, Spain, May 20-21, 2021*. IEEE, 406–410. <https://doi.org/10.1109/ICPC52881.2021.00048>
- [50] Klaus U. Schulz. 1990. Makanin’s Algorithm for Word Equations - Two Improvements and a Generalization. In *Word Equations and Related Topics, First International Workshop, IWWERT ’90, Tübingen, Germany, October 1-3, 1990, Proceedings (Lecture Notes in Computer Science, Vol. 572)*, Klaus U. Schulz (Ed.). Springer, 85–150. https://doi.org/10.1007/3-540-55124-7_4
- [51] Helgi Sigurbjarnarson, James Bornholt, Emina Torlak, and Xi Wang. 2017. Push-Button Verification of File Systems via Crash Refinement. In *Proceedings of the 2017 USENIX Annual Technical Conference, USENIX ATC 2017, Santa Clara, CA, USA, July 12-14, 2017*, Dilma Da Silva and Bryan Ford (Eds.). USENIX Association. <https://www.usenix.org/conference/atc17/technical-sessions/presentation/sigurbjarnarson>
- [52] Gert Smolka and Ralf Treinen. 1994. Records for Logic Programming. *J. Log. Program.* 18, 3 (1994), 229–258. [https://doi.org/10.1016/0743-1066\(94\)90044-2](https://doi.org/10.1016/0743-1066(94)90044-2)
- [53] Snyk Security Team. 2018. *Zip Slip: Arbitrary File Write Through Archive Extraction*. Technical Report. Snyk. <https://res.cloudinary.com/snyk/image/upload/v1528192501/zip-slip-vulnerability/technical-whitepaper.pdf>
- [54] Jean-Pierre Talpin and Pierre Jouvelot. 1994. The Type and Effect Discipline. *Inf. Comput.* 111, 2 (1994), 245–296. <https://doi.org/10.1006/INCO.1994.1046>
- [55] Balder ten Cate and Luc Segoufin. 2010. Transitive closure logic, nested tree walking automata, and XPath. *J. ACM* 57, 3 (2010), 18:1–18:41. <https://doi.org/10.1145/1706591.1706598>
- [56] Ralf Treinen. 1993. Feature Constraints with First-Class Features. In *Mathematical Foundations of Computer Science 1993, 18th International Symposium, MFCS’93, Gdansk, Poland, August 30 - September 3, 1993, Proceedings (Lecture Notes in Computer Science, Vol. 711)*, Andrzej M. Borzyszkowski and Stefan Sokolowski (Eds.). Springer, 734–743. https://doi.org/10.1007/3-540-57182-5_64
- [57] GitHub user keyvin. 2015. Moved ~/.local/share/steam. Ran steam. It deleted everything on system owned by user. #3671. <https://github.com/ValveSoftware/steam-for-linux/issues/3671>.
- [58] W3C. 2017. XML Path Language (XPath) 3.1. (2017). <https://www.w3.org/TR/2017/REC-xpath-31-20170321/>
- [59] W3C. 2017. XQuery Update Facility 3.0. (2017). <https://www.w3.org/TR/2017/NOTE-xquery-update-30-20170124/>